# Algorithms for Biological Sequence Analysis 25/26

## *Hidden Markov Models*

Miriam Santos
[dCC] @ Faculty of Sciences University of Porto

# Outline



- Introduction and Motivation
- Markov Models and Markov Chains
- Hidden Markov Models
- Dishonest Casino Example
- Finding GC-rich Regions Example
- Determining 5' Splice Site Example
- Algorithmic Settings for HMMs
- Scoring and Forward Algorithm
- Decoding and Viterbi Algorithm
- HMMER and hmmlearn

# Introduction and Motivation

- **Hidden Markov Models (HMMs)** are a fundamental tool from machine learning widely used in computational biology. Using HMMs, we can explore the underlying structure of DNA or polypeptide sequences, and detection regions of special interest.

- For instance, we can identify promoter regions, CpG islands, and other functional elements.

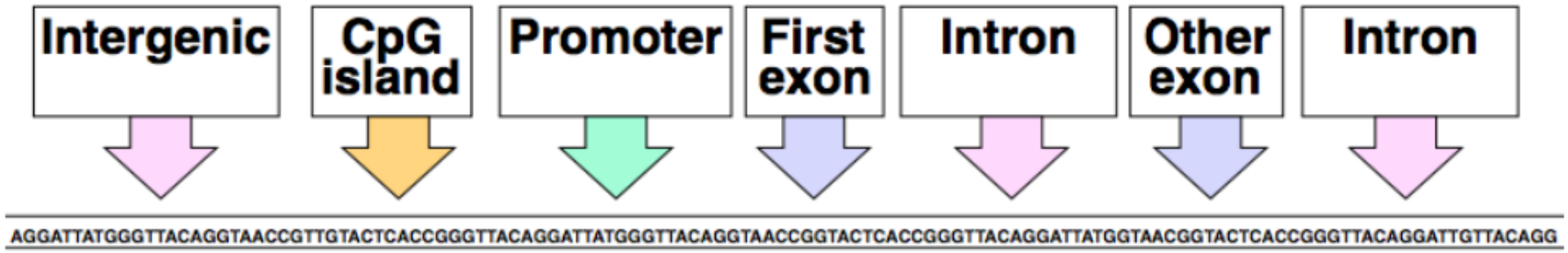
- Given a new sequence of DNA, we look for nucleotide compositions, k-mers, GC content, motifs, ORF. **We look for patterns to determine reasonable probabilitic models.**

**1. Make hypothesis.**
**2. Build a generative model to describe that hypothesis.**
**3. Use that model to find sequences of similar type.**

# Introduction and Motivation

**Such a model will allow us to:**

- Generate (***emit***) sequences of similar type according to the generative model
- Recognize the ***hidden state*** that has ***most likely generated*** the observation

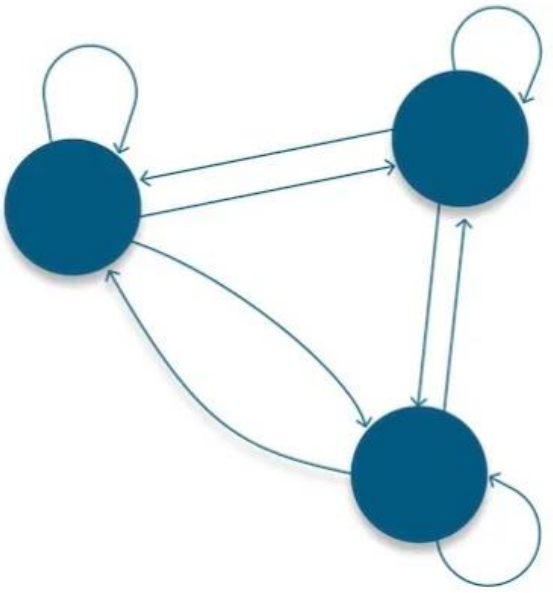



*Modeling biological sequences*

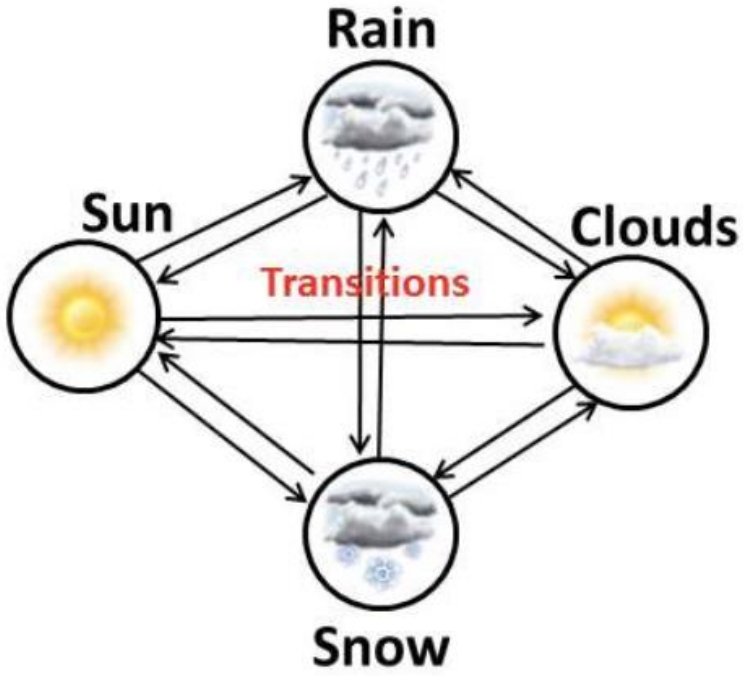
# Markov Chains and HMMs

- Markov created a way to describe how random (or "stochastic") systems or processes evolve over time.

- The system is modeled as a ***sequence of states*** and it can ***move in between states with a specific probability***. The states are ***connected***, hence, they form a ***chain***.

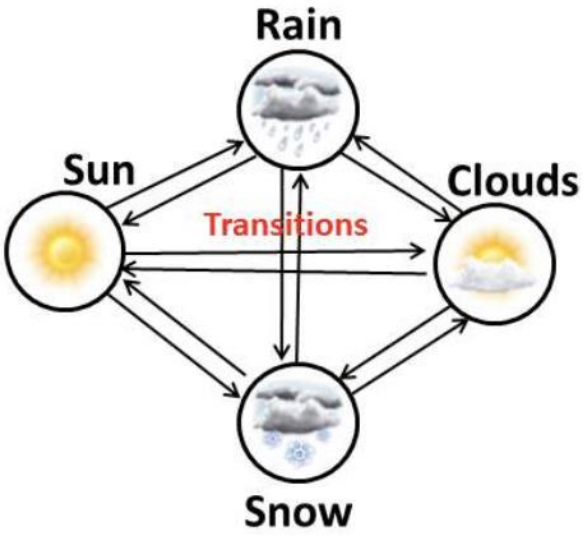


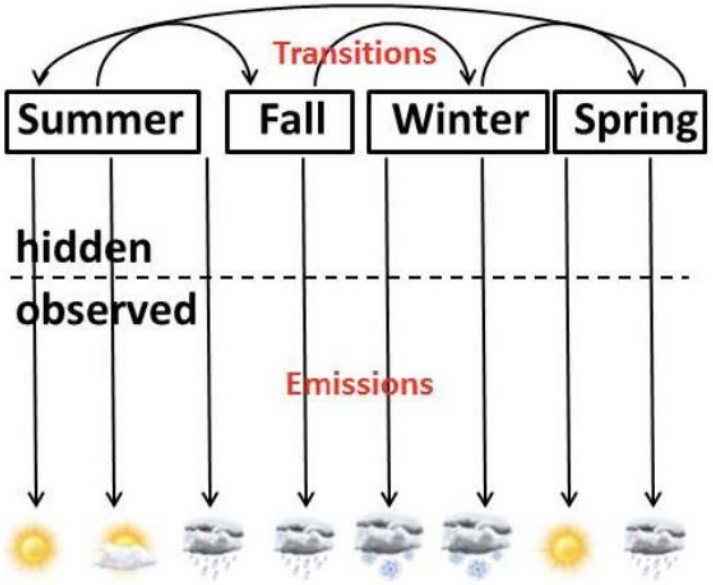
Example of a Markov chain.



# Markov Chains and HMMs

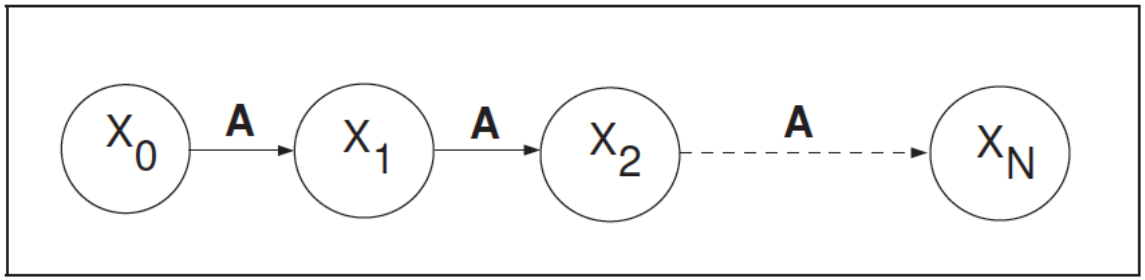
- A **Markov chain** is the simplest type of Markov model, where **all states are observable. Hidden Markov Models** have **hidden states:** you can't see them directly in the chain, only through the ***observation of another process (emission)*** that depends on it.



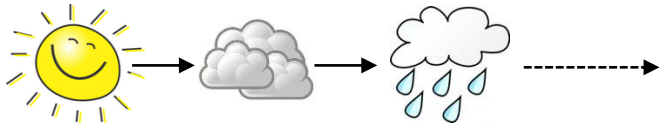



# Markov Model

- **A Markov Model** is a chain-strutured process where **future states depend only on the present state**, not on the sequence of events that preceded it.




*The X at a given time is called the state. The value of $X_n$ depends only on $X_{n-1}$*




**State**: sunny cloudy rainy sunny ?

# Markov Model



90 % sunny
10% rainy

**State :** sunny sunny rainy sunny

**State transition probability (table/graph)**

(The probability of tomorrow's weather given today's weather)

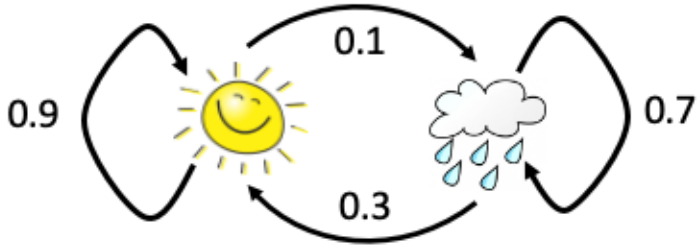
**Output format 1:**

| Today | Tomorrow | Probability |
|---|---|---|
| sunny | sunny | 0.9 |
| sunny | rainy | 0.1 |
| rainy | sunny | 0.3 |
| rainy | rainy | 0.7 |

**Output format 2:**

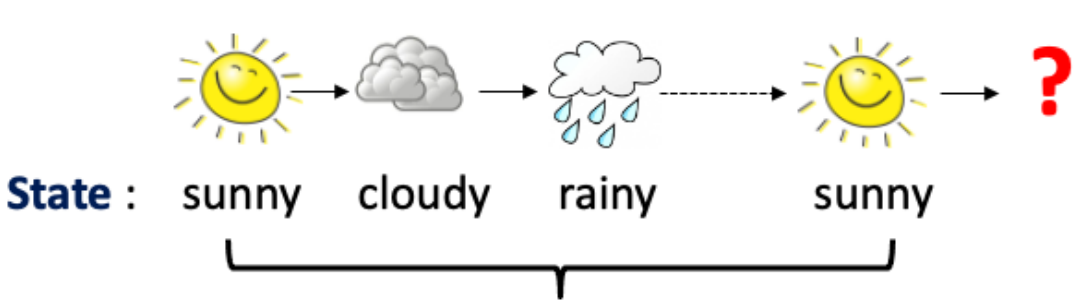
| | sunny | rainy |
|---|---|---|
| sunny | 0.9 | 0.1 |
| rainy | 0.3 | 0.7 |

**Output format 3:**



# Markov Model





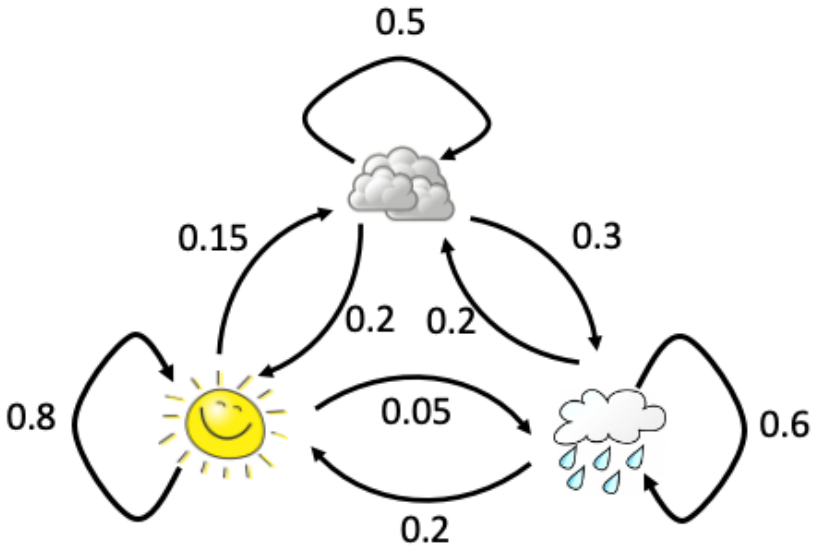

80 % sunny
15% cloudy
5% rainy

**State transition probability (table/graph)**

**Output format 1:**

| Today | Tomorrow | Probability |
|---|---|---|
| sunny | sunny | 0.8 |
| sunny | rainy | 0.05 |
| sunny | cloudy | 0.15 |
| rainy | sunny | 0.2 |
| rainy | rainy | 0.6 |
| rainy | cloudy | 0.2 |
| cloudy | sunny | 0.2 |
| cloudy | rainy | 0.3 |
| cloudy | cloudy | 0.5 |

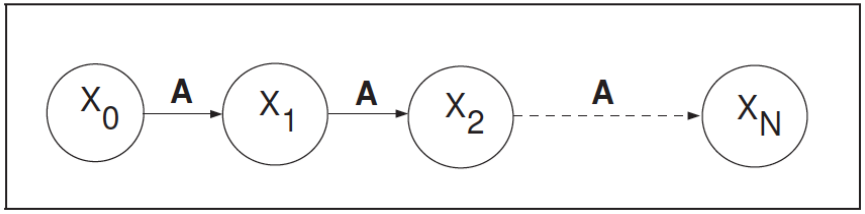
**Output format 3:**



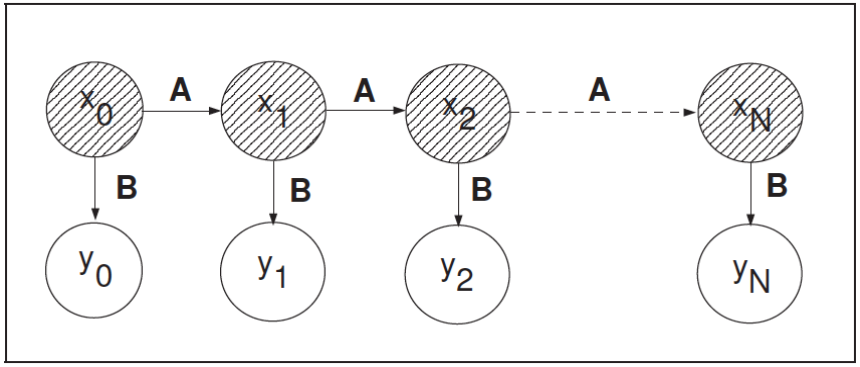
# The Hidden Markov Model

- **A Hidden Markov Model** is a Markov chain for which the state is not directly observable.

**Markov Model**




**Hidden Markov Model**



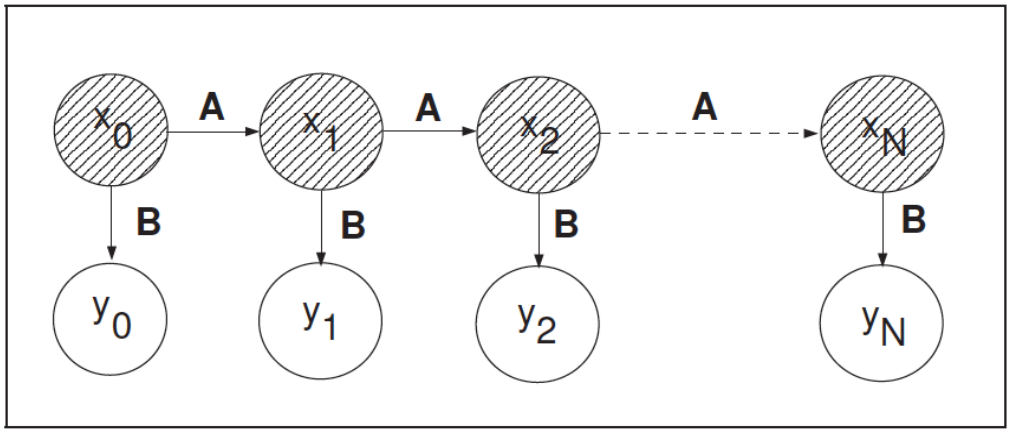

- **Hidden States:** The true states of a system that can be described by a Markov process (e.g., the **weather** / the **season**).

- **Observed States:** The states of the process that are "visible" (e.g., **umbrella** / **weather**). Sometimes called "***emissions***".

# The Hidden Markov Model

**Hidden Markov Models** are composed of the main building blocks:

- **Hidden States:** represented by $x_0, x_1, x_2, ..., x_N$
- **Transition Matrix:** represented by **A**
- **Sequence of Observations:** represented by $y_0, y_1, y_2, ..., y_N$
- **Observation Likelihood Matrix // Emission Matrix:** represented by **B**
- **Initial Probability Distribution**



# The Hidden Markov Model



| | sunny | rainy | cloudy |
|---|---|---|---|
| sunny | 0.8 | 0.05 | 0.15 |
| rainy | 0.2 | 0.6 | 0.2 |
| cloudy | 0.2 | 0.3 | 0.5 |

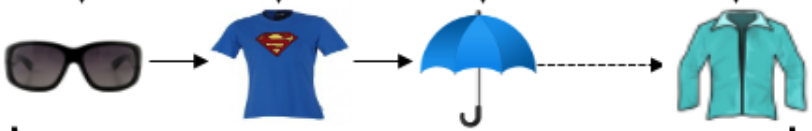
sum to 1

**State transition probability table**

**Hidden States**

***state path***

**Observed States**



***emissions***

**State emission probability table**

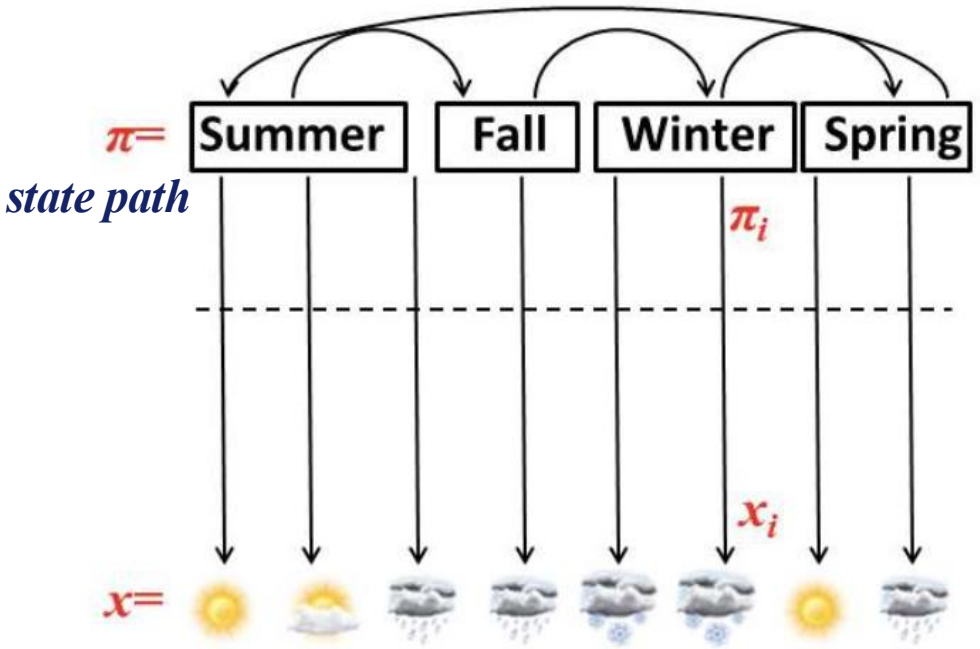
| | sunglasses | T-shirt | umbrella | Jacket |
|---|---|---|---|---|
| sunny | 0.4 | 0.4 | 0.1 | 0.1 |
| rainy | 0.1 | 0.1 | 0.5 | 0.3 |
| cloudy | 0.2 | 0.3 | 0.1 | 0.4 |

sum to 1

*The probability of observing a particular observable state given a particular hidden state.*

# The Hidden Markov Model

- **There's an invisible process and an observable process.** The invisible process is the Markov Chain, chaining together multiple hidden states that are traversed over time in order to reach an outcome. HMMs describe the evolution of **observable events,** which themselves are dependent on internal (*hidden*) factors that cannot be directly observed.

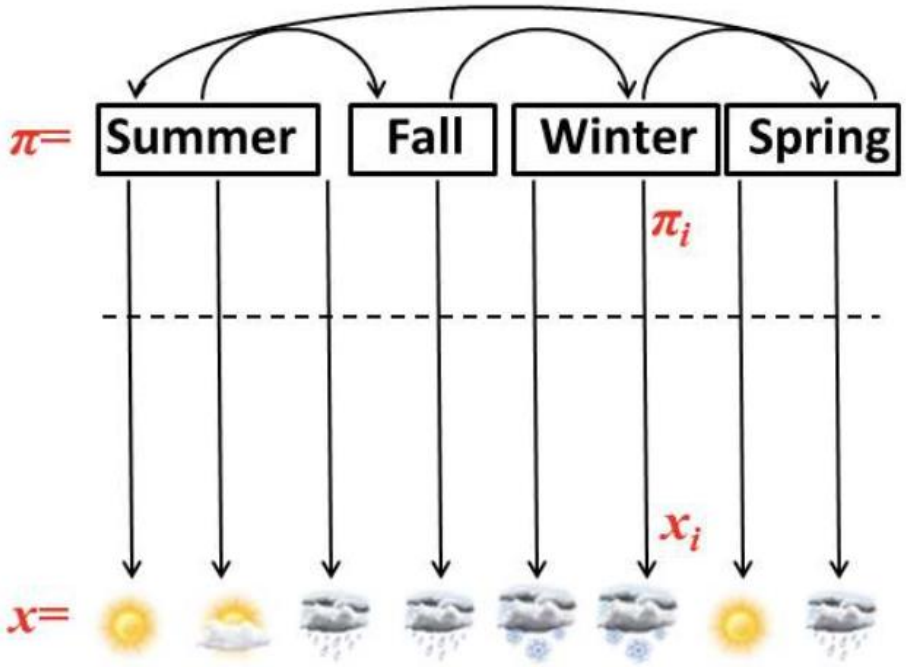



Transitions: $a_{kl}=P(\pi_i=l|\pi_{i-1}=k)$
Transition probability from state $k$ to state $l$

Emissions: $e_k(x_i)=P(x_i|p_i=k)$
Emission probability of symbol $x_i$ from state $k$

# The Hidden Markov Model

- x represents the ***sequence of observations***
- π represents the ***hidden path***
- Each entry $a_{kl}$ of **Transition Matrix A** denotes the probability of transition from state k to state l
- Each entry $e_k(x_i)$ of the **emission vector/matrix** denotes the probability of observing $x_i$ from state k




**Transitions:** $a_{kl}=P(\pi_i=l|\pi_{i-1}=k)$
Transition probability from state $k$ to state $l$

**Emissions:** $e_k(x_i)=P(x_i|p_i=k)$
Emission probability of symbol $x_i$ from state $k$

# Formalizing Markov Chains



- A Markov Chain is given by a **finite set of states and transition probabilities** between the states. The probability of transitioning to a state **depens only on the current state**, and is independent of how the current state was reached.

- It can be defined as a triple (Q, p, A): a set of states **Q**, a transition matrix **A**, a vector **p** of initial probabilities.

- The key property of Markov Chains is that they are *memory-less*: each state depends **only** on the previous state, and therefore:

$$P(x_i \mid x_{i-1}, \ldots, x_1) = P(x_i \mid x_{i-1})$$

- The **probability of a sequence of states** can be decomposed as:

$$P(x) = P(x_L, x_{L-1}, \ldots, x_1) = P(x_L|x_{L-1})P(x_{L-1}|x_{L-2})\ldots P(x_2|x_1)P(x_1)$$

P(xL) is calculated from transition probabilities. If we multiply the initial state probabilities at time *t=0* by the transition matrix A, we get the probabilities at time *t=1*. Multuplying by the appropriate power AL of the transition matrix, we obtain the state probabilities at time *t =L*.

# Formalizing Hidden Markov Models



- HMMs model a system in which observations appear as results of states that we cannot observe directly.

- These observations, or *emisions*, result from a particular state based on a set of probabilities (called "emission probabilities").

- Formally, a HMM is a 5-tuple (Q, p, A, **V, E**): a set of states **Q**, a transition matrix **A**, a vector **p** of initial probabilities, **a set of observed symbols V (*emissions*) and a matrix of emission probabilties, E.**

- **For example, V = {A, C, T, G} or V = set of 20 aminoacids**

- **Regarding E, for each state s in Q, the emission probability is defined as $e_{sk} = P(v_k \mid s)$. That is, the probability of observing symbol $v_k$ given state s.**

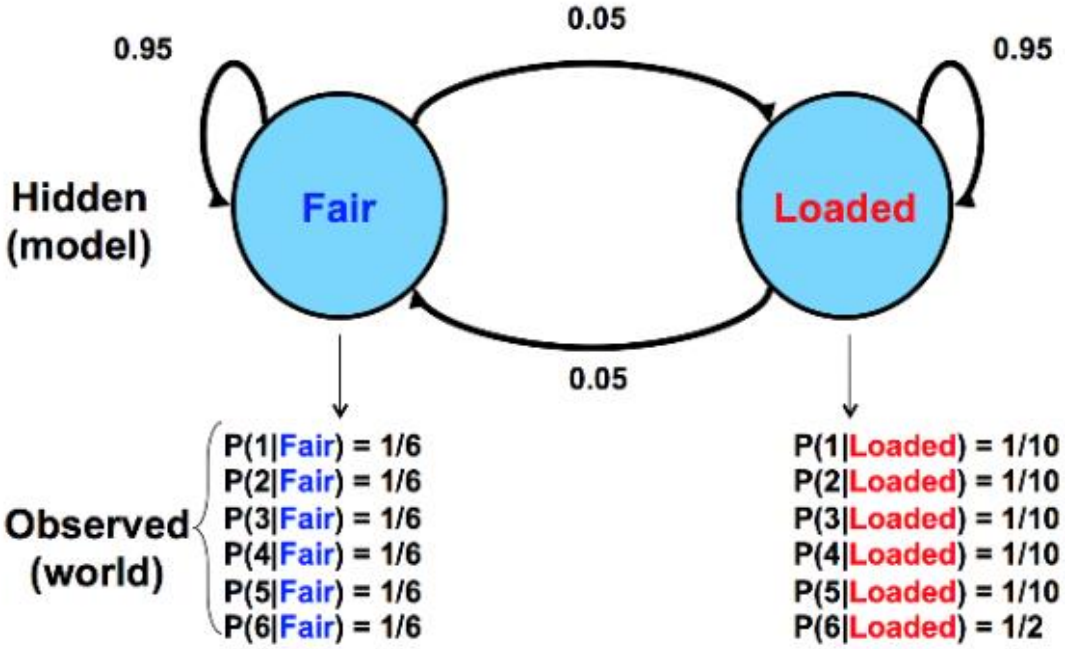
# The Dishonest Casino



**A dishonest casino has 2 types of dice:**

- **Fair die:** P(1) = P(2) = P(3) = P(4) = P(5) = P(6) = 1/6
- **Loaded die:** P(1) = P(2) = P(3) = P(4) = P(5) = 1/10 and P(6) = ½

The dealer can switch between the two at any time without us knowing. The only information we have are the rolls that we observe. We can represente the state of the casino die with a simple Markov Model.




***Hidden and Observed States?***

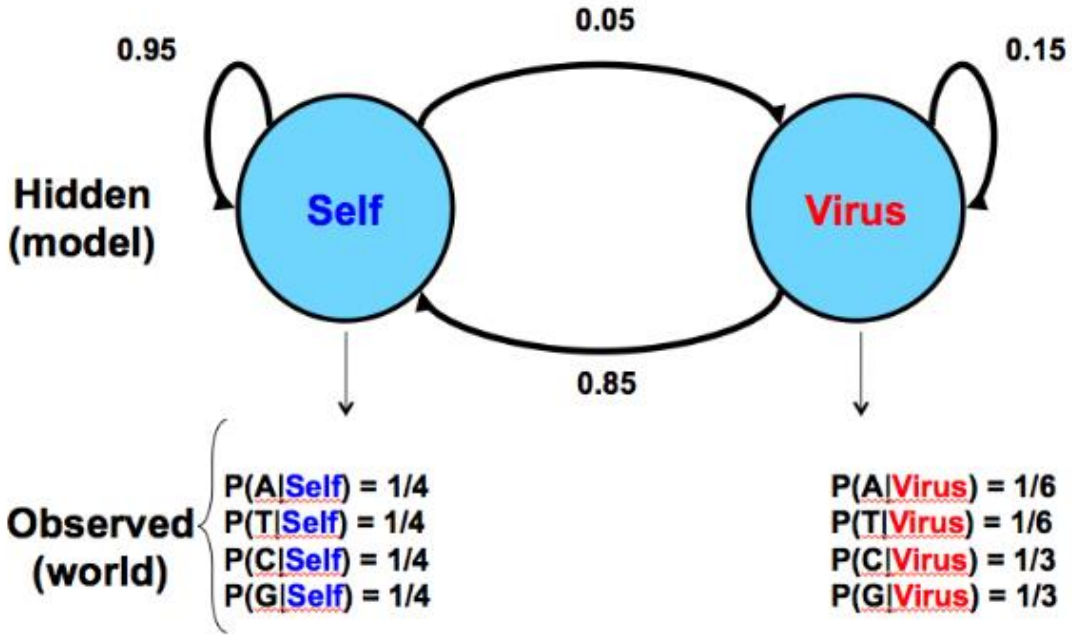
***Transition Matrix?***

***Emission Matrix?***

# An analogy with Biology



- A sequence of DNA that has two potential sources: injection by a **virus** versus **normal** production by the organism itself.




- Given this model as hypothesis, we would observe the frequencies of C and G to give us clues regarding the source of the sequence in question.

- This model assumes that **viral inserts will have higher CpG prevalence**, which leads to higher probabilities of C and G occurence.

# The Dishonest Casino



**We are at the casino and observe this sequence of rolls:**

We would like to know whether it is more likely that the casino is using the **fair die or the loaded die.**

**We will consider two possible sequences:**

- Always using a **fair** die.
- Always using a **loaded** die.

# The Dishonest Casino

## Using a FAIR die:

In the first case, where we assume the dealer is always using a fair die, the transition and emission probabilities are shown in Figure 7.7. The probability of this sequence of states and observed emissions is a product of terms which can be grouped into three components: 1/2, the probability of starting with the fair die; $(1/6)^{10}$, the probability of the sequence of rolls if we always use the fair die; and lastly $(0.95)^9$, the probability that we always continue to use the fair die.
In this model, we assume $\pi = F, F, F, F, F, F, F, F, F, F$, and we observe $x = 1, 2, 1, 5, 6, 2, 1, 6, 2, 4$

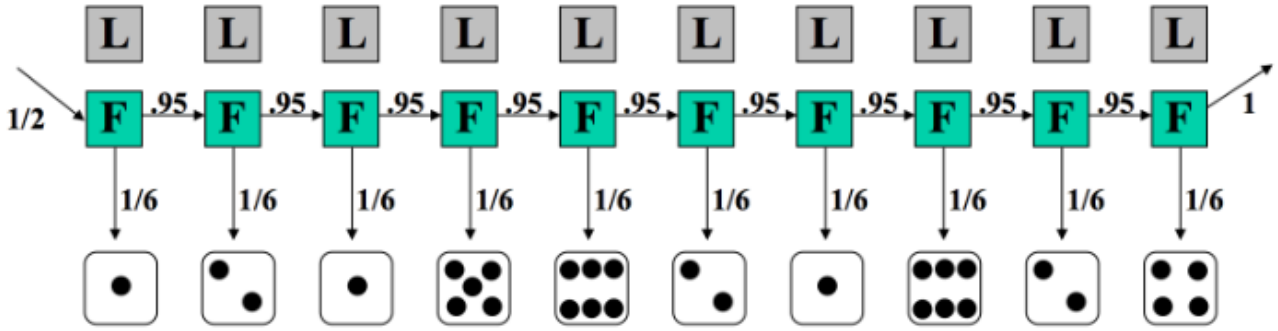



Figure 7.7: Running the model: probability of a sequence, given path consists of all fair dice

Now we can calculate the joint probability of x and π as follows:

$$\begin{aligned} P(x, \pi) &= P(x \mid \pi) P(\pi) \\ &= \frac{1}{2} \times P(1 \mid F) \times P(F \mid F) \times P(2 \mid F) \cdots \\ &= \frac{1}{2} \times \left(\frac{1}{6}\right)^{10} \times (0.95)^9 \\ &= 5.2 \times 10^{-9} \end{aligned}$$

# The Dishonest Casino

## Is it more likely than using a LOADED die?

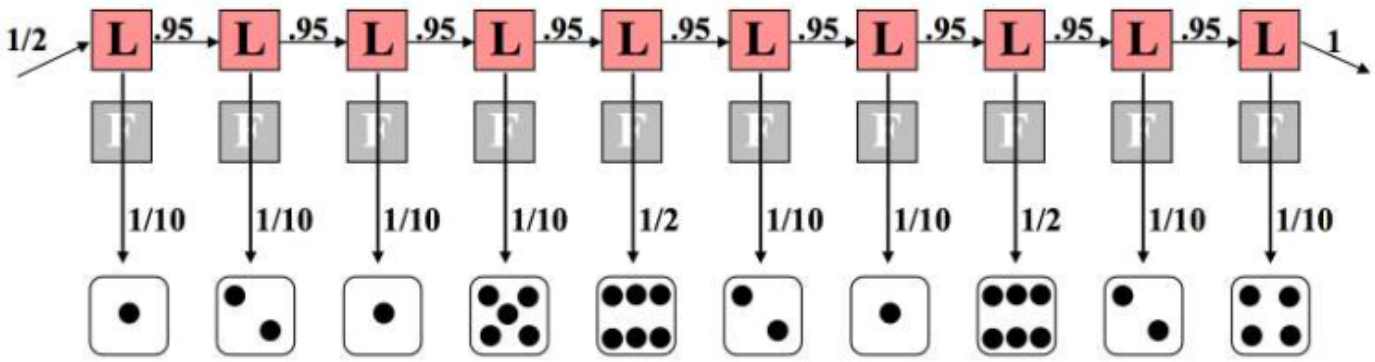



Figure 7.8: Running the model: probability of a sequence, given path consists of all loaded dice

Let us consider the opposite extreme where the dealer always uses a loaded die, as depicted in Figure 7.8. This has a similar calculation except that we note a difference in the emission component. This time, 8 of the 10 rolls carry a probability of 1/10 because the loaded die disfavors non-sixes. The remaining two rolls of six have each a probability of 1/2 of occurring. Again we multiply all of these probabilities together according to principles of independence and conditioning. In this case, the calculations are as follows:

$$\begin{aligned} P(x,\pi) &= \frac{1}{2} \times P(1 \mid L) \times P(L \mid L) \times P(2 \mid L) \cdots \\ &= \frac{1}{2} \times \left(\frac{1}{10}\right)^{8} \times \left(\frac{1}{2}\right)^{2} \times (0.95)^{9} \\ &= 7.9 \times 10^{-10} \end{aligned}$$

# The Dishonest Casino

## Now the dealer can switch die at some point during the sequence:

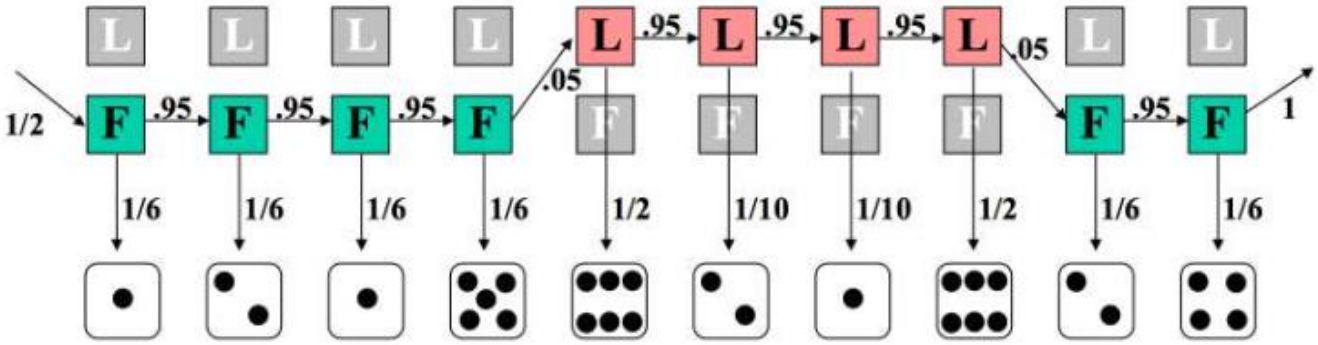



Figure 7.9: Partial runs and die switching

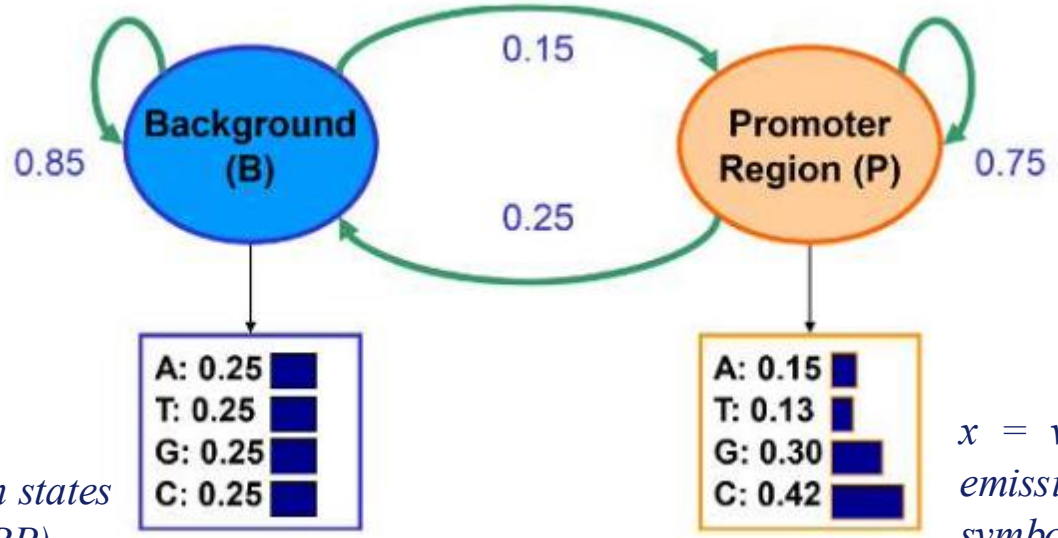
Again, we can calculate the likelihood of the joint probability of this sequence of states and observations. Here, six of the rolls are calculated with the fair die, and four with the loaded one. Additionally, not all of the transition probabilities are 95% anymore. The two swaps (between fair and loaded) each have a probability of 5%.

$$\begin{aligned} P(x,\pi) &= \frac{1}{2} \times P(1 \mid L) \times P(L \mid L) \times P(2 \mid L) \cdots \\ &= \frac{1}{2} \times \left(\frac{1}{10}\right)^2 \times \left(\frac{1}{2}\right)^2 \times \left(\frac{1}{6}\right)^6 \times (0.95)^7 \times (0.05)^2 \\ &= 4.67 \times 10^{-11} \end{aligned}$$

# Finding GC-rich regions



- Finding CG rich regions by modeling DNA sequences drawn from 2 distributions: **background** and **promotor**.
- By looking at a sequence, we want to identify which regions originate from a background distribution and which regions are from a promotor region.




*π = vector of hidden states in a path (e.g., BPPBP).*

*x = vector of observable emissions consisting of symbols {A, T, G, C}*

| | *B* | *P* |
|---|---|---|
| *B* | *0.85* | *0.15* |
| *P* | *0.25* | *0.75* |

***Transition Matrix***

| | *A* | *T* | *G* | *C* |
|---|---|---|---|---|
| *B* | *0.25* | *0.25* | *0.25* | *0.25* |
| *P* | *0.15* | *0.13* | *0.30* | *0.42* |

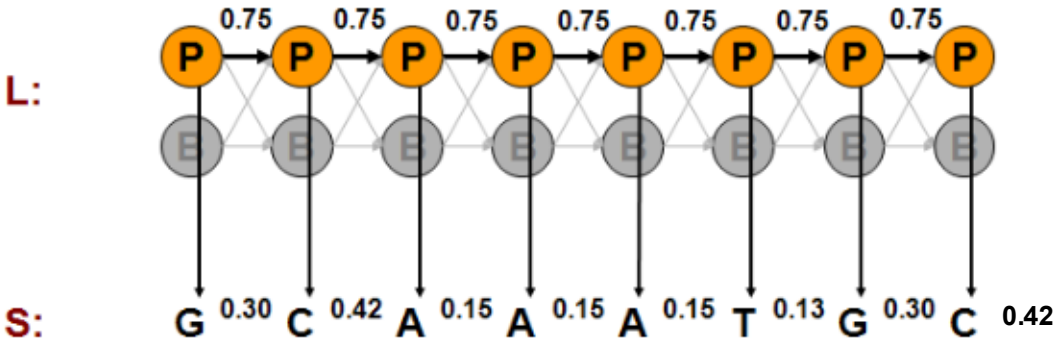
***Emission Matrix***

# Finding GC-rich regions



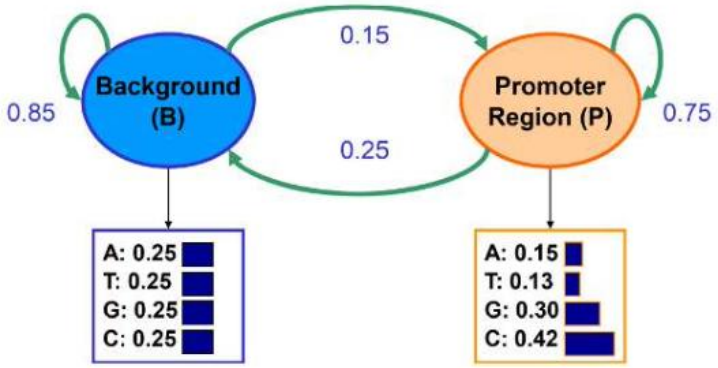
*We will start in state B with high probability (1) since most locations do not correspond to promotor genes. We then draw an emission from P(x|B), where each nucleotide occurs with probability 0.25. Now we have that the probability of remaining in state B is 0.85 and the probability of transitioning to state P is 0.15, and so on...*

***If path is all promoter (P)***




$$P(x,\pi)=a_p*e_p(G)*a_{pp}*e_p(G)*a_{pp}*e_p(C)*a_{pp}*e_p(A)*a_{pp}*\ldots$$
$$=a_p*(0.75)^7*(0.15)^3*(0.13)^1*(0.30)^2*(0.42)^2$$
$$=9.3*10^{-7}$$

***P(seq AND path) = P(x, π) = P(x|π) * P(π)***




***Transition Matrix***

| | B | P |
|---|---|---|
| B | 0.85 | 0.15 |
| P | 0.25 | 0.75 |

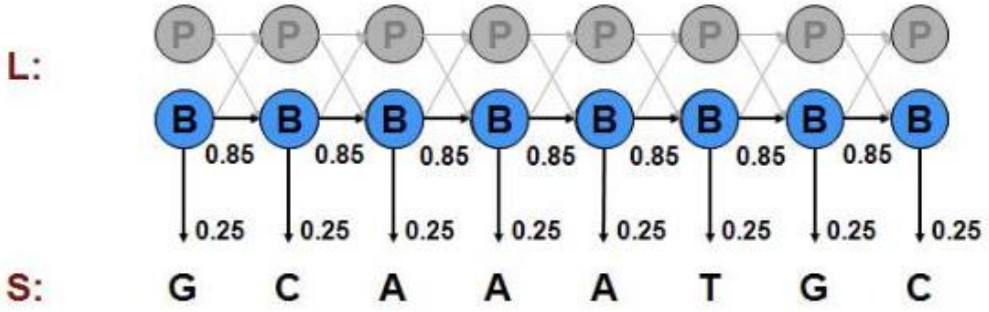
***Emission Matrix***

| | A | T | G | C |
|---|---|---|---|---|
| B | 0.25 | 0.25 | 0.25 | 0.25 |
| P | 0.15 | 0.13 | 0.30 | 0.42 |

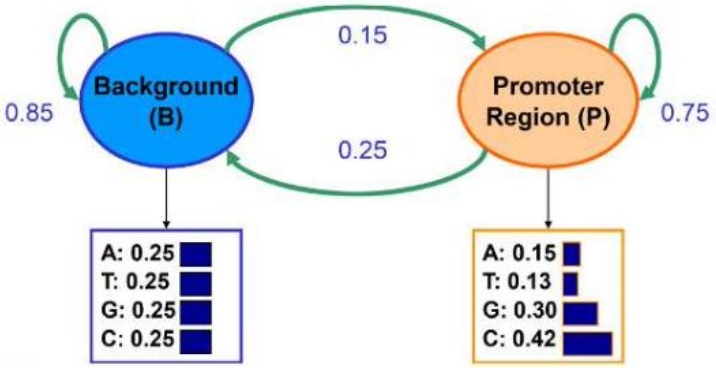
# Finding GC-rich regions



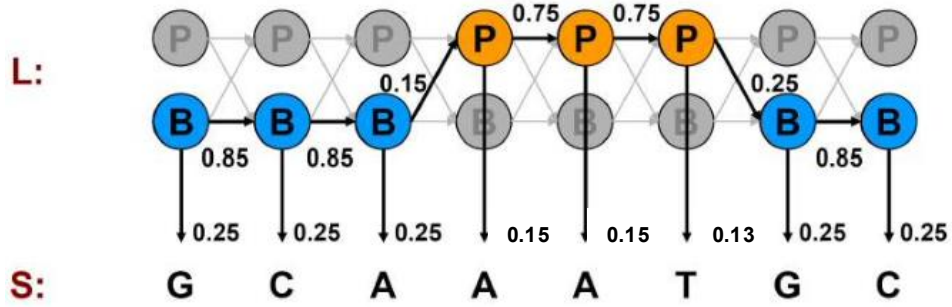
*If path is all background (B)*




$$P = P(G|B)P(B_1|B_0)P(C|B)P(B_2|B_1)P(A|B)P(B_3|B_2)\ldots P(C|B_7)$$
$$= (0.85)^7 \times (0.25)^8$$
$$= 4.9\times10^{-6}$$




*If path sequence is mixed (P and B)*




$$P(x,\pi) = (0.85)^3 * (0.25)^6 * (0.75)^2 * (0.15)^3 * 0.13$$
$$= 3.7 * 10^{-8}$$

*Transition Matrix*

| | B | P |
|---|---|---|
| B | 0.85 | 0.15 |
| P | 0.25 | 0.75 |

*Emission Matrix*

| | A | T | G | C |
|---|---|---|---|---|
| B | 0.25 | 0.25 | 0.25 | 0.25 |
| P | 0.15 | 0.13 | 0.30 | 0.42 |

# Finding GC-rich regions



- Based on our calculations P(x, π) for 3 possible paths:

$$P(x, \pi_B) = 4.9 \times 10^{-6}$$

$$P(x, \pi_P) = 9.3 \times 10^{-7}$$

$$P(x, \pi_{\text{mixed}}) = 3.7 \times 10^{-8}$$

- The ***pure-background*** alternative is the most likely option of the possibilities we have examined.

# Determining 5' Splice Site



- **Markov Models** are probabilistic processes **whose probabilities depend only on the current state of the systems**, not on any of the previous states.

- In **Hidden Markov Models**, we *infer* a series of states that are **not directly observable.**

- **In Bioinformatics, Hidden Markov Models are often used when:**
  The ***observable*** states are sequences (e.g., DNA, Aminoacids);
  The ***hidden*** states are the underlying structure or organization of the system;

- We'll consider na example by Sean R. Eddy to infer, based on a DNA sequence, where within a particular gene, we transition from an exon, to a 5' splice, to an intron.

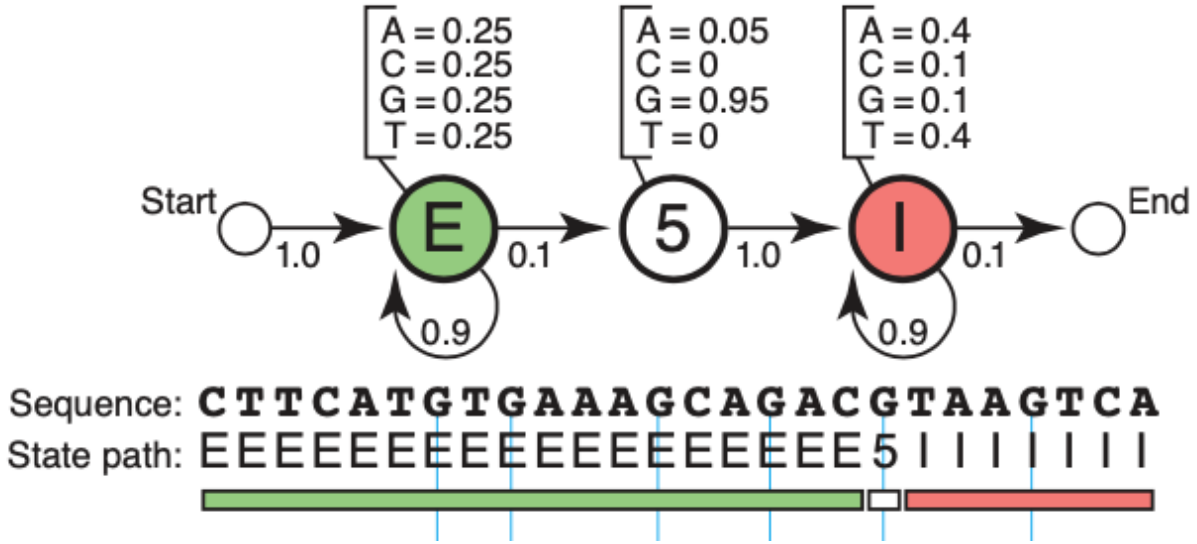
*Eddy, S. What is a hidden Markov model?. Nat Biotechnol **22**, 1315–1316 (2004). https://doi.org/10.1038/nbt1004-1315*

# Determining 5' Splice Site



- We'll consider na example by Sean R. Eddy to infer, based on a **DNA sequence**, where within a particular gene, we **transition from an *exon*, to a 5' splice, and then to an *intron*.**

- **Hidden layer:** *sequence of states (exon - 5' splice – intron)*
- **Observable layer:** *DNA sequence*

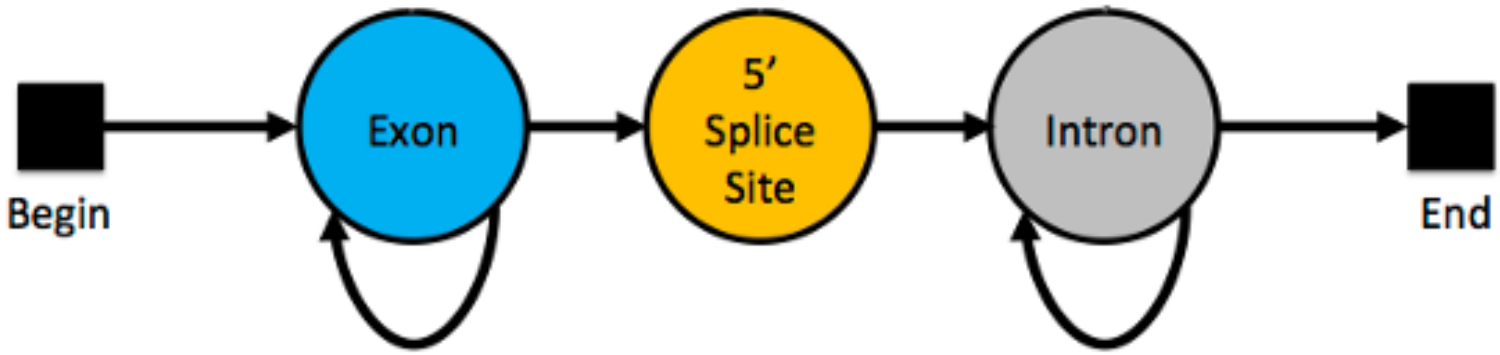



*Eddy, S. What is a hidden Markov model?. Nat Biotechnol **22**, 1315–1316 (2004). https://doi.org/10.1038/nbt1004-1315*

# Determining 5' Splice Site



- We assume ***our first nucleotide in in an exon***, and we may ***continue in an exon for an unknown number of nucleotides***, before we move on to a 5' splice site.
- We assume that the ***5' splice site is exactly 1-nucleotide long***. This way, it doesn't loop back to itself and directly moves to the *intron*.
- We ***can stay at the intron also for an unknown number of nucleotides***, before we reach the end of the DNA read.

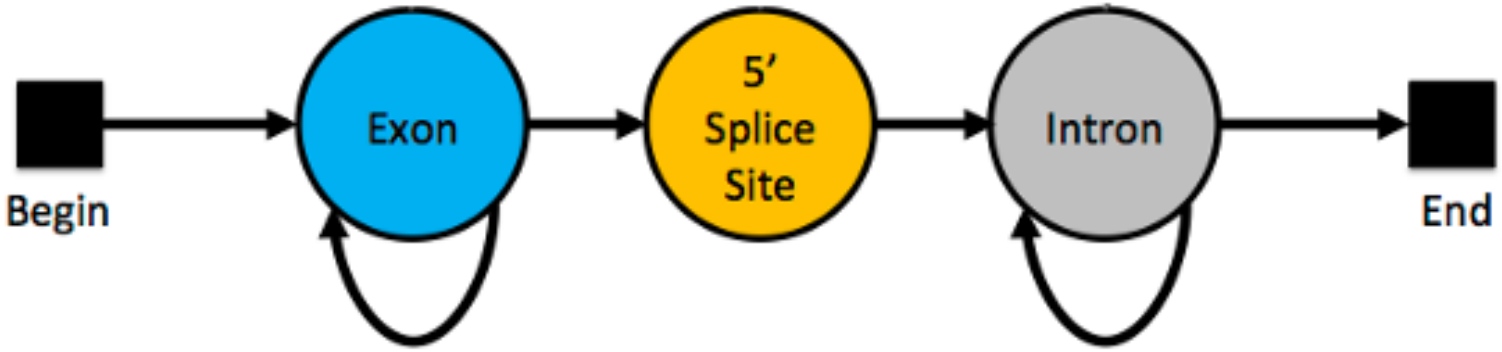


# Determining 5' Splice Site



- To describe this model, we need an **Initialization Vector**
- **Initialization Vector = Initial Probability Distributions**
- It describes the probabilities of *starting in diferent states!*

## State Model




## Initialization Vector

*Probabilities that our first nucleotide is in an exon, 5', intron?*

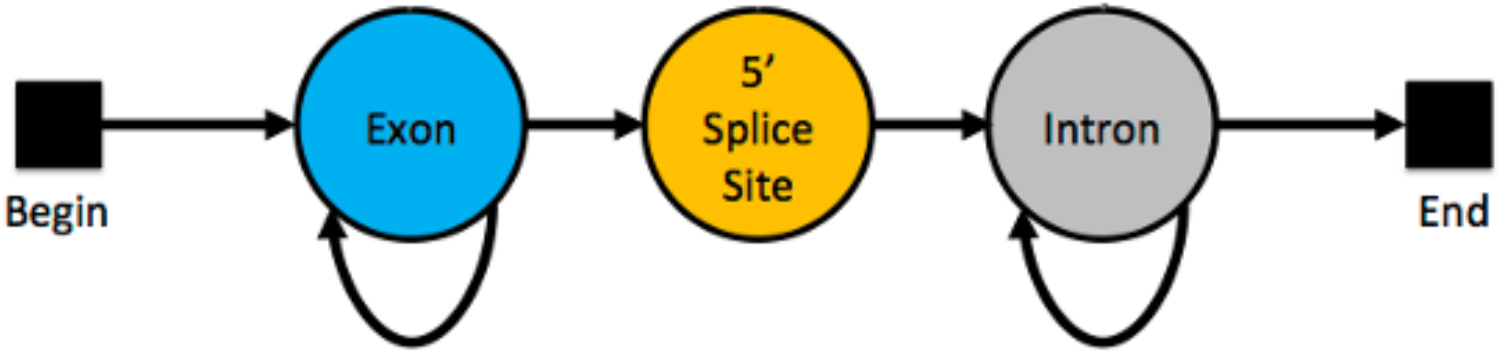
$$P(start\_in\_exon) =$$
$$P(start\_in\_5') =$$
$$P(start\_in\_intron) =$$

# Determining 5' Splice Site



- To describe this model, we need an **Initialization Vector**
- **Initialization Vector = Initial Probability Distributions**
- It describes the probabilities of *starting in diferent states!*

## State Model




*Probabilities that our first nucleotide is in an exon, 5', intron?*

$$P(start\_in\_exon) = 1$$
$$P(start\_in\_5') = 0$$
$$P(start\_in\_intron) = 0$$

*In this model, some states will never be the starting state in the Markov Chain (their initial probability is zero).* ***In all cases, sum of initial probabilities needs to add up to 1!***

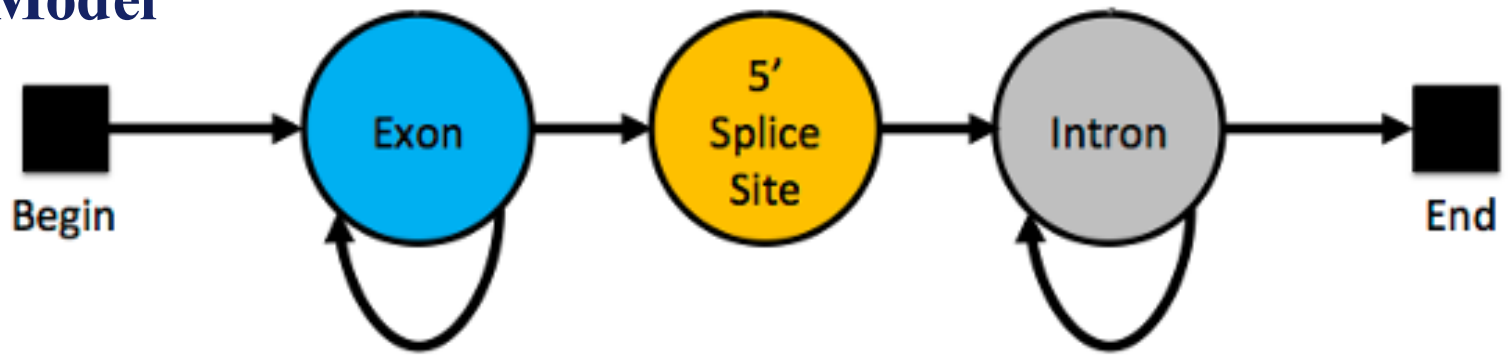
**Initialization Vector = [1 0 0]**

# Determining 5' Splice Site



- **Transition Matrix:** It describes, for a given nucleotide, the *probability of being in a given state, based on the state we were before.*

## State Model




## Transition Matrix

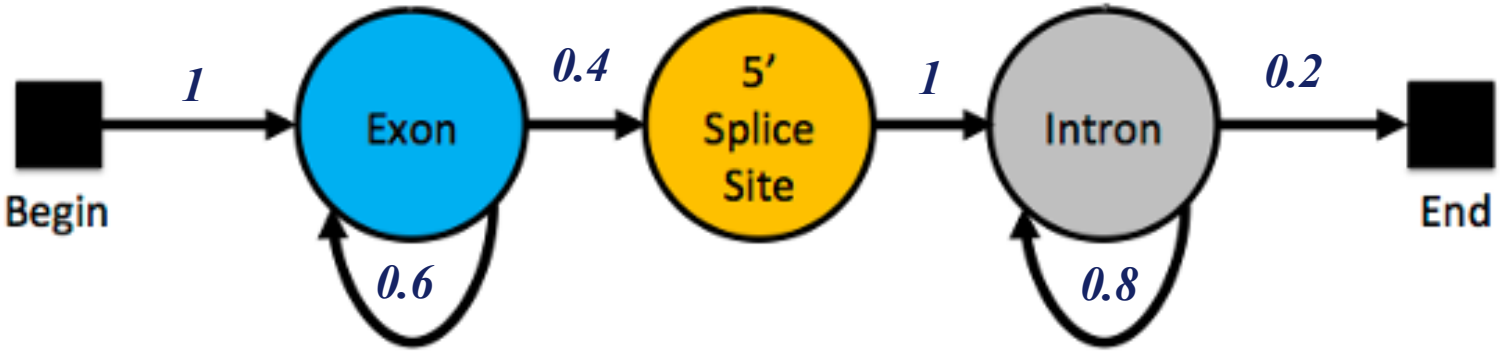
*Which of these can you already fill in?*

| *From:* \ *To:* | *Exon* | *5' SS* | *Intron* | *End* |
|---|---|---|---|---|
| *Exon* | | | | |
| *5' SS* | | | | |
| *Intron* | | | | |

# Determining 5' Splice Site



- **The *other probabilities* (*non-zero in this example*) will depend on our biological knowledge!**




## Transition Matrix

| | *Exon* | *5' SS* | *Intron* | *End* |
|---|---|---|---|---|
| *Exon* | *0.6* | *0.4* | *0* | *0* |
| *5' SS* | *0* | *0* | *1* | *0* |
| *Intron* | *0* | *0* | *0.8* | *0.2* |

- The probabilities in **each row need to add up to 1**!
- *(Are all exons/introns going to be the same length?)*

# Determining 5' Splice Site



- **Emission Matrix:** The probability that, if we're on an exon, 5'SS or intron, that we will emmit a particular nucleotide (A, C, T, G).
- **Emission Matrix = Observation Likelihood Matrix**
- Again, this will be based on our biological knowledge of exons, 5'SS and introns. They can depend on the gene, gene regions, or particular organisms!

## Emission Matrix

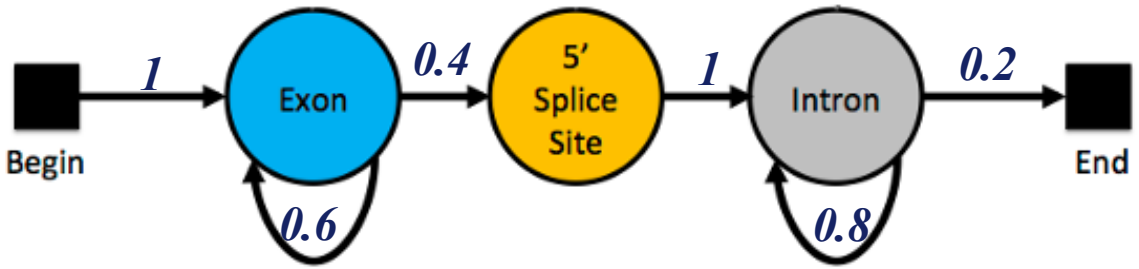
| | *A* | *C* | *G* | *T* | |
|---|---|---|---|---|---|
| ***Exon*** | *0.25* | *0.25* | *0.25* | *0.25* | *Guess: uniform base sequences?* |
| ***5' SS*** | *0* | *0* | *1* | *0* | *Assumption: 5'SS in G (Eddy's)* |
| ***Intron*** | *0.4* | *0.1* | *0.1* | *0.4* | *Biological: Normally A/T rich* |

# Determining 5' Splice Site



- Based on a particular DNA Sequence, **what are all the possible state paths?**

A - G - G - T




- **State Path 1:**

A - G - G - T

E - 5 - I - I

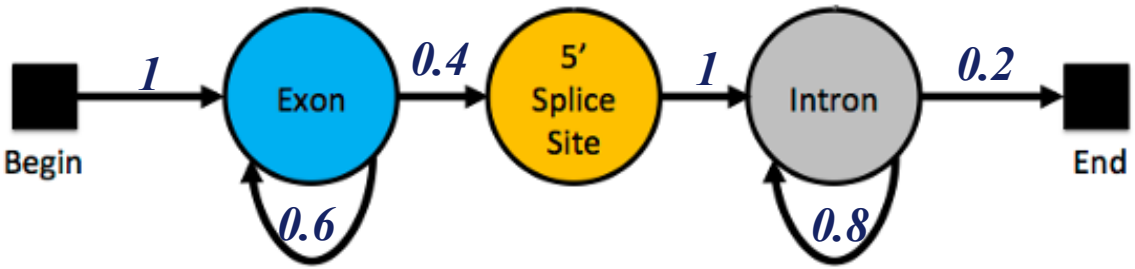
- **State Path 2?**

# Determining 5' Splice Site



- Based on a particular DNA Sequence, **what are all the possible state paths?**

A - G - G - T




- **State Path 1:**

A - G - G - T

E - 5 - I - I - *end*

- **State Path 2:**

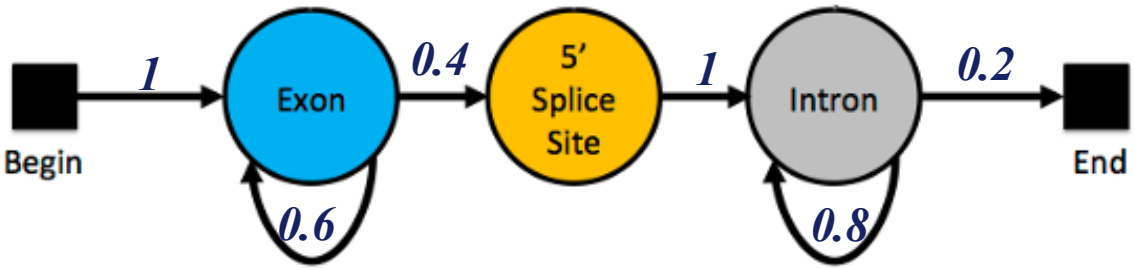
A - G - G - T

E - E - 5 - I - *end*

# Determining 5' Splice Site

- Based on a particular DNA Sequence, **what are all the possible state paths?**

A - G - G - T




**State Path 1**

A - G - G - T

E - 5 - I - I - *end*

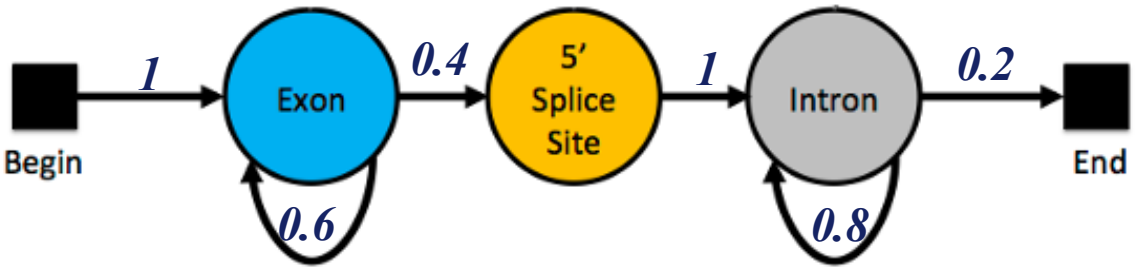
$P(state\_path\_1) =$

# Determining 5' Splice Site



- Based on a particular DNA Sequence, **what are all the possible state paths?**

A - G - G - T




**State Path 1**

A - G - G - T

E - 5 - I - I - *end*

$P(state\_path\_1) =$ 1 // 0.4 // 1 // 0.8 // 0.2

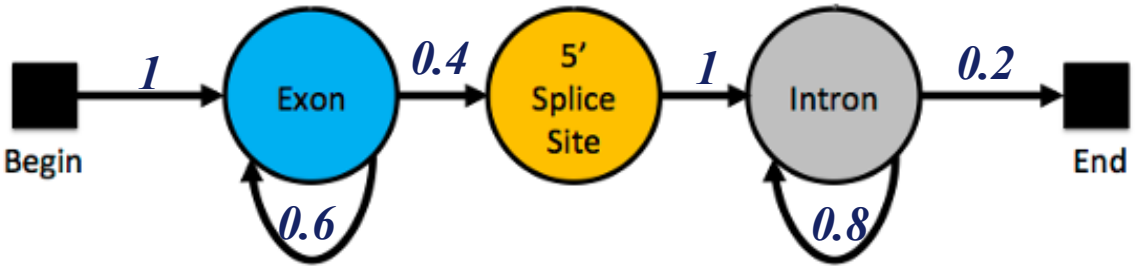
- Because this is a Markov Models, these **probabilities are independent** of each other, and we can **combine them by multiplication**!

# Determining 5' Splice Site



- Based on a particular DNA Sequence, **what are all the possible state paths?**

A - G - G - T




**State Path 1**

A - G - G - T

E - 5 - I - I - *end*

$$P(state\_path\_1) = 1 \times 0.4 \times 1 \times 0.8 \times 0.2 = 0.064$$

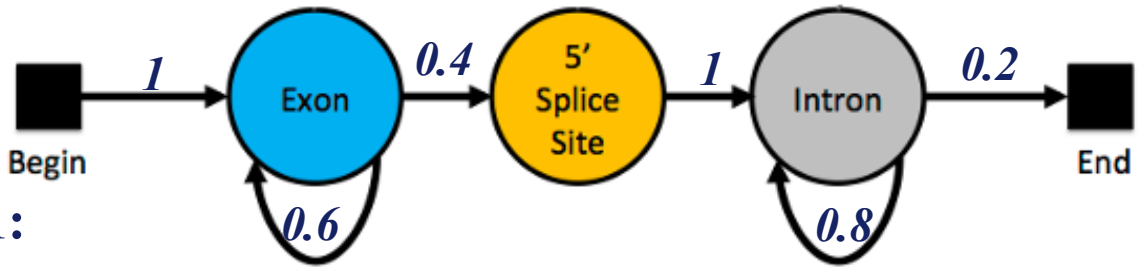
- What is the probability of state path 2, **P(state_path_2)**?

# Determining 5' Splice Site



- Based on a particular DNA Sequence, **what are all the possible state paths?**

A - G - G - T




- **State Path 1:**

A - G - G - T

E - 5 - I - I - *end*

1 // 0.4 // 1 // 0.8 // 0.2

$$P(state\_path\_1) = 1 \times 0.4 \times 1 \times 0.8 \times 0.2 = 0.064$$

- **State Path 2:**

A - G - G - T

E - E - 5 - I - *end*

1 // 0.6 // 0.4 // 1 // 0.2

$$P(state\_path\_2) = 1 \times 0.6 \times 0.4 \times 1 \times 0.2 = 0.048$$

# Determining 5' Splice Site



- We calculated the probabilities of **getting specific state paths.**
- But **if I had a specific state path**, "E-5-I-I-end", **what is the probability that it emits sequence** "AGGT"?
- **That's a conditional probability!**

$$P(obs\_sequence|state\_path\_1) =$$

*Probability that we observe a certain sequence, **given that** we are on state path 1.*

***The same as P(A\|B), "probability of A given B".***

**State Path 1**

A - G - G - T

E - 5 - I - I - *end*

*If we are in an exon, what is the probability that it emits nucleotide A?*

**Emission Matrix**

| | *A* | *C* | *G* | *T* |
|---|---|---|---|---|
| ***Exon*** | ***0.25*** | ***0.25*** | ***0.25*** | ***0.25*** |
| ***5' SS*** | ***0*** | ***0*** | ***1*** | ***0*** |
| ***Intron*** | ***0.4*** | ***0.1*** | ***0.1*** | ***0.4*** |

# Determining 5' Splice Site



**State Path 1**

A - G - G - T

E - 5 - I - I - *end*

*If we are in an exon, what is the probability that it emits nucleotide A?* **0.25**

**Emission Matrix**

| | *A* | *C* | *G* | *T* |
|---|---|---|---|---|
| ***Exon*** | ***0.25*** | ***0.25*** | ***0.25*** | ***0.25*** |
| ***5' SS*** | ***0*** | ***0*** | ***1*** | ***0*** |
| ***Intron*** | ***0.4*** | ***0.1*** | ***0.1*** | ***0.4*** |

$$P(obs\_sequence|state\_path\_1) = 0.25 \times 1 \times 0.1 \times 0.4 = 0.01$$

**State Path 2:**

A - G - G - T

E - E - 5 - I - *end*

$$P(obs\_sequence|state\_path\_2) = 0.25 \times 0.25 \times 1 \times 0.4 = 0.025$$

# Determining 5' Splice Site



- At the moment, we have calculated the **probability of a getting paths 1 and 2**:

$$P(state\_path\_1) = 1 \times 0.4 \times 1 \times 0.8 \times 0.2 = 0.064$$

$$P(state\_path\_2) = 1 \times 0.6 \times 0.4 \times 1 \times 0.2 = 0.048$$

- And the **probability that we get the observed sequence, given those possible hidden paths**:

$$P(obs\_sequence|state\_path\_1) = 0.25 \times 1 \times 0.1 \times 0.4 = 0.01$$

$$P(obs\_sequence|state\_path\_2) = 0.25 \times 0.25 \times 1 \times 0.4 = 0.025$$

- **We can now combine both probabilities to determine:**

$$P(state\_path\_1 \cap obs\_sequence) = P(state\_path\_1\ AND\ obs\_sequence)$$

$$P(state\_path\_2 \cap obs\_sequence) = P(state\_path\_2\ AND\ obs\_sequence)$$

*Probability that we followed state path 1/2 AND given that we did that, we observed the given DNA sequence.*

# Determining 5' Splice Site

- **We can now combine both probabilities to determine:**

$$P(state\_path\_1 \cap obs\_sequence) = P(state\_path\_1\ AND\ obs\_sequence)$$

$$P(state\_path\_2 \cap obs\_sequence) = P(state\_path\_2\ AND\ obs\_sequence)$$

***Conditional Probability***

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

***same as***

$$P(B \cap A)$$

***A*** = *"observing sequence x"*
***B*** = *"observing state k"*

$$P(A \cap B) = P(B) \times P(A|B)$$

# Determining 5' Splice Site

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \qquad P(A \cap B) = P(B) \times P(A|B)$$

***A*** = *"observing sequence x"*
***B*** = *"observing state k"*

- **I can now combine both probabilities to determine:**

$$P(state\_path\_1 \cap obs\_sequence) = P(state\_path\_1 \ AND \ obs\_sequence)$$
$$P(state\_path\_2 \cap obs\_sequence) = P(state\_path\_2 \ AND \ obs\_sequence)$$

$$P(state\_path\_1) = 0.064 \qquad P(obs\_sequence|state\_path\_1) = 0.01$$
$$P(state\_path\_2) = 0.048 \qquad P(obs\_sequence|state\_path\_2) = 0.025$$

- **Which yields:**

$$P(state\_path\_1 \cap obs\_sequence) = 0.064 \times 0.01 = 0.00064$$
$$P(state\_path\_2 \cap obs\_sequence) = 0.048 \times 0.025 = 0.0012$$

# Determining 5' Splice Site



- Sometimes, we are interested in determininig P(B|A) instead.

$$P(B|A) = \frac{P(B \cap A)}{P(A)} \qquad P(\pi|x) = \frac{P(x,\pi)}{P(x)}$$

*This would answer the question:*
*"Based on the observed sequence x, what is the likelihood of state k?"*
*OR, better yet:*
***"Given this sequence, what is the most likely sequence of hidden states that produced it?"***

$$P(x) = \sum_{\pi} P(x,\pi)$$

$$P(\pi|x) = \frac{P(x,\pi)}{P(x)} = \frac{P(x,\pi)}{\sum_{\pi} P(x,\pi)}$$

# Determining 5' Splice Site



$$P(B|A) = \frac{P(B \cap A)}{P(A)}$$

***A*** = *"observing sequence x"*
***B*** = *"observing state k"*

$$P(\pi|x) = \frac{P(x,\pi)}{P(x)} = \frac{P(x,\pi)}{\sum_{\pi} P(x,\pi)}$$

$$P(B|A) = \frac{P(B \cap A)}{P(A)} = \frac{P(B \cap A)}{\sum_{\text{all paths}} P(B \cap A)}$$

$$P(state\_path\_1|obs\_sequence) = \frac{0.00064}{0.00064 + 0.0012} = 0.348 = 34.8\%$$

$$P(state\_path\_2|obs\_sequence) = \frac{0.0012}{0.00064 + 0.0012} = 0.652 = 65.2\%$$

***Which state path is the most likely to generate the observed sequence?***

# Finding GC-rich regions

- Recall the "GC-rich regions" example, where we calculated P(x, π) for 3 possible paths:

$$P(x, \pi_B) = 4.9 \times 10^{-6}$$

$$P(x, \pi_P) = 9.3 \times 10^{-7}$$

$$P(x, \pi_{\text{mixed}}) = 3.7 \times 10^{-8}$$

- The ***pure-background*** alternative is the most likely option of the possibilities we have examined.

- **But how do we know whether it is the option most likely out of all possible paths of states to have generated the observed sequence?**

# Algorithmic Settings for HMMs



**But how do we know whether it is the option most likely out of all possible paths of states to have generated the observed sequence?**

- The brute-force approach is to examine all paths, try all possibilities and calculate their joint probabilities $P(x,\pi)$.
- Computing this sum requires considering an exponential number of possible paths.
- Besides modeling and allowing to run simulations, HMMs allow us to answer diferent questions regarding the behaviour of the systems. Most often, these questions fall onto one of these categories: **scoring**, **decoding**, and **learning**.
- For instance, computing $P(x, \pi)$ is a **scoring** task. Computing $P(x)$ over all possible paths is also a **scoring** task. Finding the "*most likely path*" is a **decoding** task.

# Algorithmic Settings for HMMs

**We can use HMMs for three types of operations/tasks:**

- **Scoring (or Likehood):** Determining the probability P(x, π) of observing a sequence, considering ***one*** specific path. We did this plently in the previous exemples.

- **Decoding:** Determining the best sequence of states that generated a specific observation. This means finding π*, the ***most likely path***, i.e., the path with maximum probability. We find $\pi^* = \text{argmax}_\pi P(x,\pi)$.

| | One path | All paths |
|---|---|---|
| Scoring | 1. Scoring x, one path<br>$P(x,\pi)$<br>Prob of a path, emissions | 2. Scoring x, all paths<br>$P(x) = \Sigma_\pi P(x,\pi)$<br>Prob of emissions, over all paths |
| Decoding | 3. Viterbi decoding<br>$\pi^* = \text{argmax}_\pi P(x,\pi)$<br>Most likely path | 4. Posterior decoding<br>$\pi^\wedge = \{\pi_i \mid \pi_i = \text{argmax}_k \Sigma_\pi P(\pi_i = k \mid x)\}$<br>Path containing the most likely state at any time point. |
| Learning | 5. Supervised learning, given π<br>$\Lambda^* = \text{argmax}_\Lambda P(x,\pi\mid\Lambda)$<br>6. Unsupervised learning.<br>$\Lambda^* = \text{argmax}_\Lambda \max_\pi P(x,\pi\mid\Lambda)$<br>Viterbi training, best path | 6. Unsupervised learning<br>$\Lambda^* = \text{argmax}_\Lambda \Sigma_\pi P(x,\pi\mid\Lambda)$<br>Baum-Welch training, over all path |

Figure 7.15: The six algorithmic settings for HMMS

- **Learning:** Determining the parameters of the HMM that led to observing a given sequence, that traversed a specific set of states.

# Scoring

**Scoring over a single path:**

- We have computed this for the dishonest casino, rich CG regions, and 5'SS determination.
- The single path calculation is essentially the likelihood of observing the given sequence over a particular path.

$$P(x, \pi) = P(x|\pi) \times P(\pi)$$

**Input**

- A sequence of observations x = [$x_1$, $x_2$, $x_3$ ...] generated by a HMM(Q, A, p, V, E) and a path of state $\pi = \pi_1, \pi_2, \pi_3$ ....

**Output**

- Joint probability P(x,π) of observing x if the hidden state sequence path is π.

# Scoring

**Scoring over all paths:**

- We use this score when we are interested in knowing the **likelihood of a particular sequence for a given HMM**. It returns the likelihood of the observed sequence under the model, i.e., summing the joint probability over every possible hidden state sequence that could have generated x.

$$P(x) = \sum_{\pi} P(x, \pi)$$

**Input**

- A sequence of observations x = [$x_1$, $x_2$, $x_3$ ...] generated by a HMM(Q, A, p, V, E).

**Output**

- Joint probability P(x,π) of observing x over all posible sequences of hidden states π.

# Forward Algorithm

**Scoring over all paths:**

- A given sequence x can be generated by many paths. We can determine P(x) as the probability of that sequence being generated, over all possible paths that could generate it.

- The question is "*how likely is the sequence given the model*"?

$$P(x) = \sum_{\pi} P(x, \pi)$$

- **The challenge of obtaining this probability is that there are too many paths.**

- However, we can use **dynamic programming** to calculate this sum iteratively, using the **Forward Algorithm**.

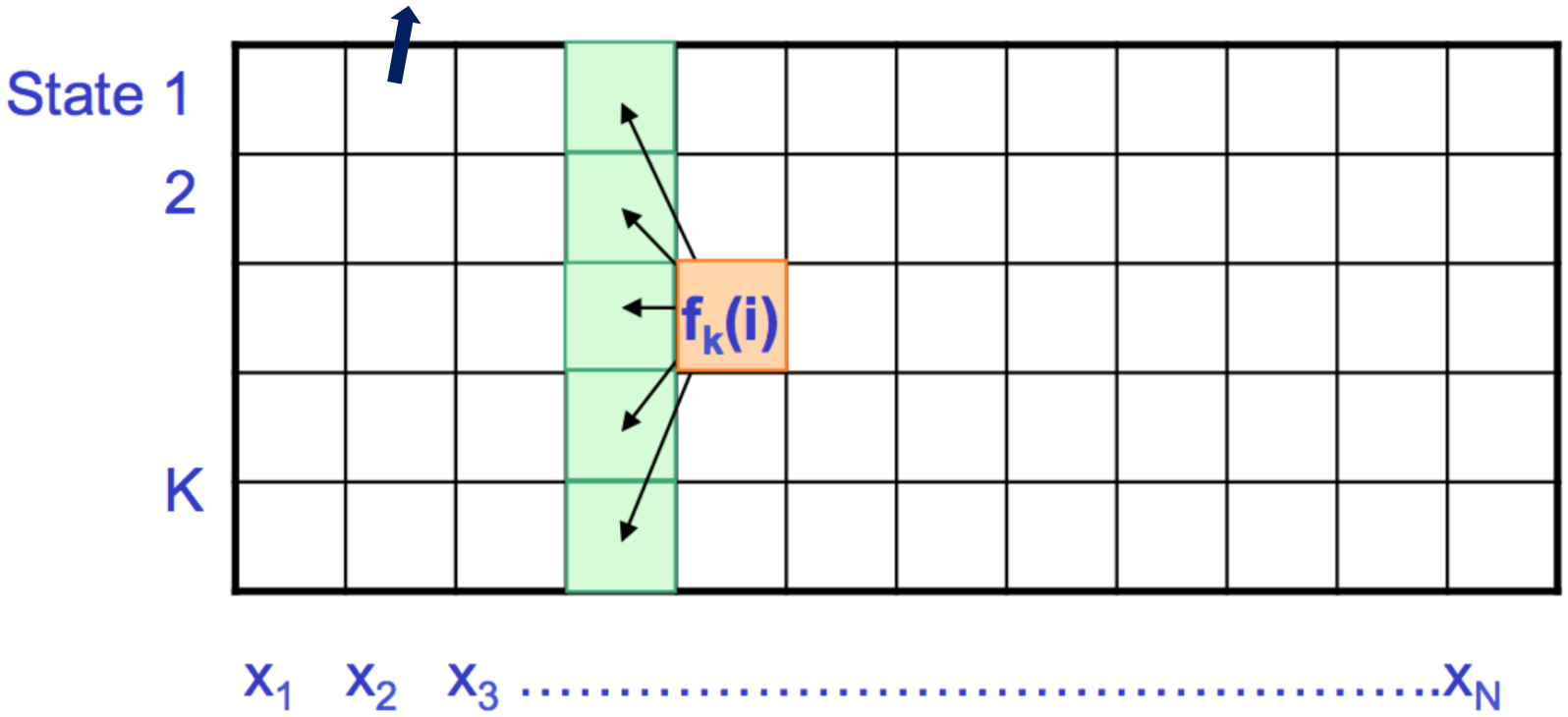
# Forward Algorithm



The **Forward Algorithm** is a dynamic programming approach to find P(x, π) over all paths by computing iteratively $f_k(i)$ as:

$$f_k(i) = e_k(x_i) \sum_j f_j(i-1) a_{jk}$$

*Total probability of all paths that produce observations up to i and end in state k. The value of each cell is computed by summing over the probabilities of every path that could leads us to this cell.*




*$e_k$ is the emission probability for state k of emiting $x_i$ and $a_{jk}$ is the probability of transitioning from state j to state k*

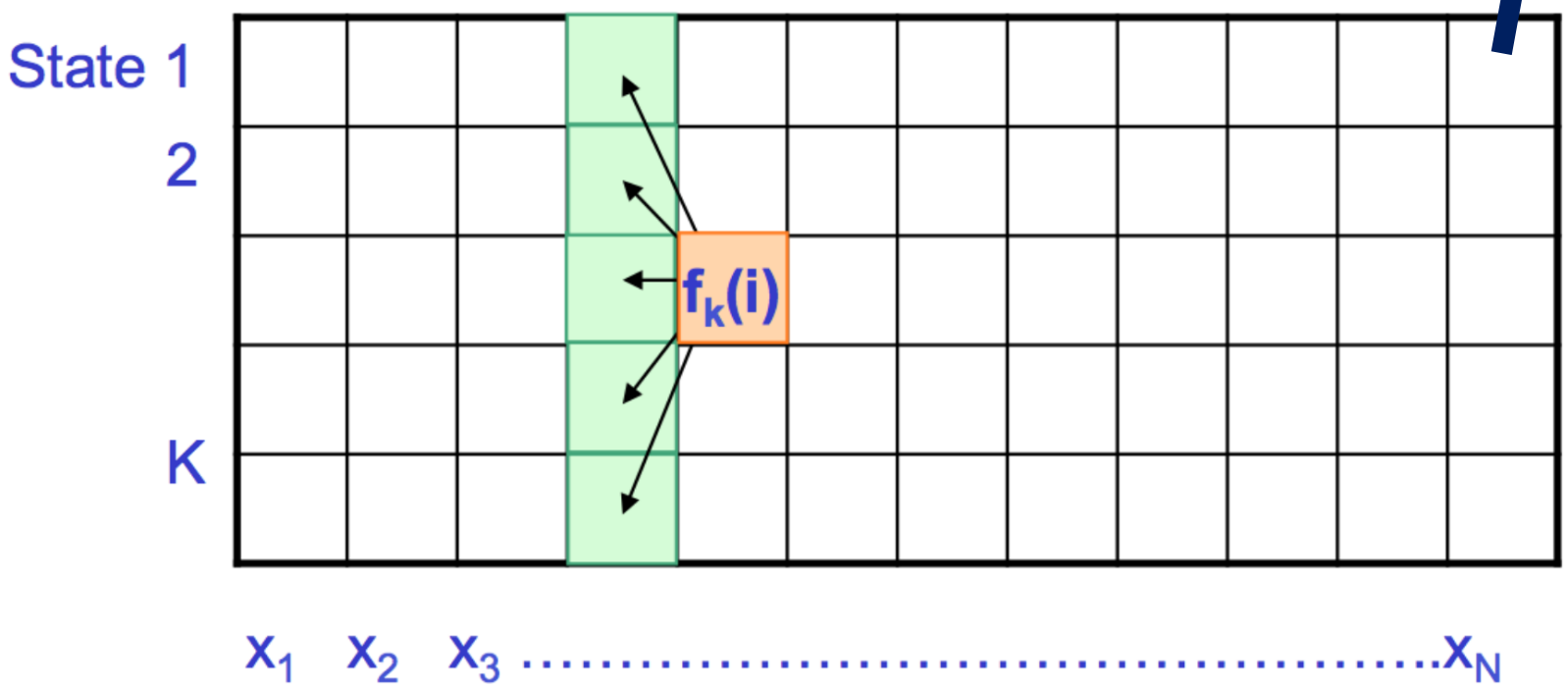
*K is the number of stattes and N is the length of the observed sequence*

# Forward Algorithm



- Initialization $(i = 0) : f_0(0) = 1, f_k(0) = 0 \text{ for } k > 0$
- Iteration $(i = 1 \ldots N) : f_k(i) = e_k(x_i) \sum_j f_j(i-1) a_{jk}$
- Termination: $P(x, \pi^*) = \sum_k f_k(N)$

*The sum of the elements in the last column of the DP matrix provides the total probability of an observed sequence of characters.*




*$e_k$ is the emission probability for state k of emiting $x_i$ and $a_{jk}$ is the probability of transitioning from state j to state k*

# Forward Algorithm



- Initialization $(i = 0)$ : $f_0(0) = 1$, $f_k(0) = 0$ for $k > 0$
- Iteration $(i = 1 \ldots N)$ : $f_k(i) = e_k(x_i) \sum_j f_j(i-1) a_{jk}$
- Termination: $P(x, \pi^*) = \sum_k f_k(N)$

*Before the sequence starts, we are at the start state with probability 1*

*Before the sequence starts, we are not yet in any state*

*The first "real column" is computed using the normal recurrence. For i = 1, we use $f_j(0)$ which is 1, so $fk(1) = e_k(x_1)a_{0k}$, which is the emission * the probability of starting at state k.*

# Forward Algorithm Example



$$f_k(i) = e_k(x_i) \sum_j f_j(i-1) a_{jk}$$

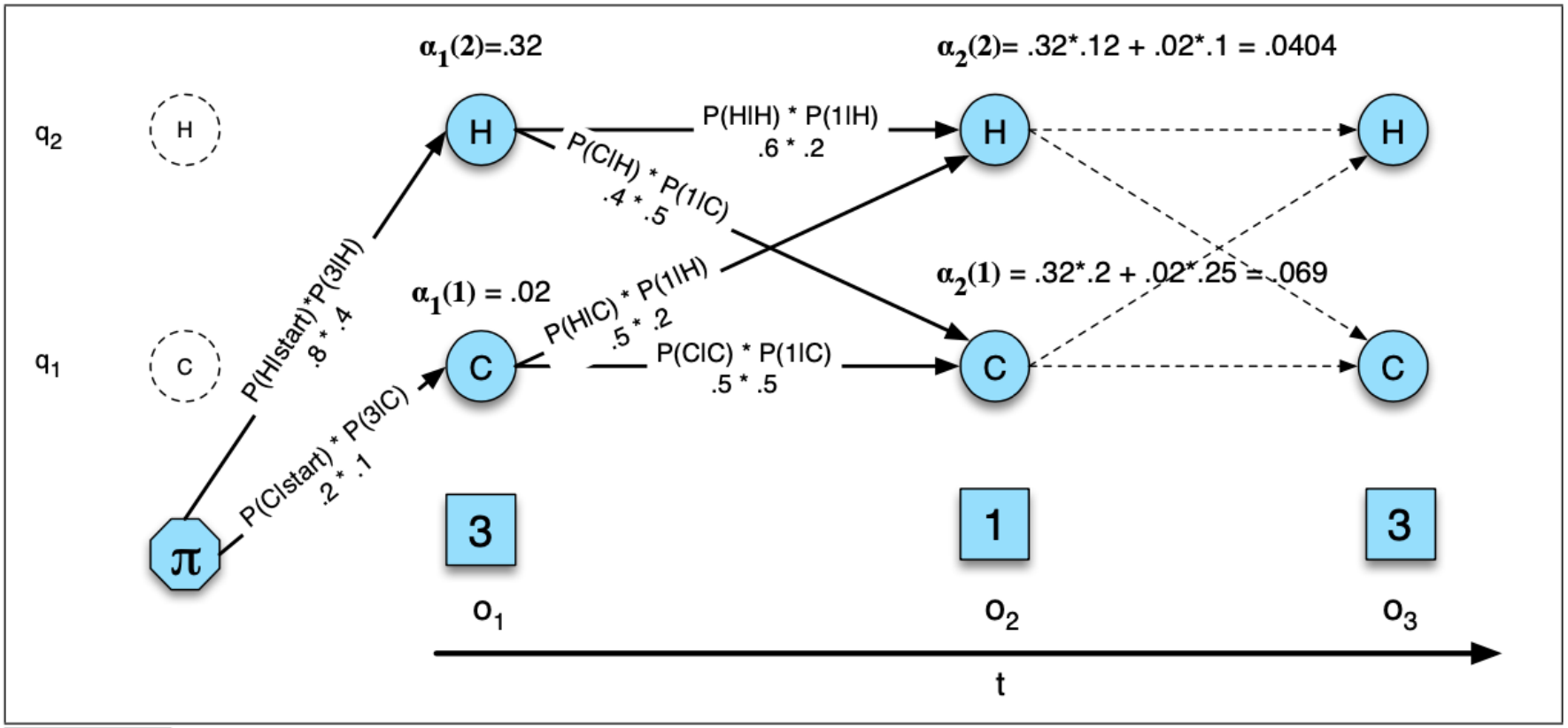



**Figure A.5** The forward trellis for computing the total observation likelihood for the ice-cream events *3 1 3*. Hidden states are in circles, observations in squares. The figure shows the computation of $\alpha_t(j)$ for two states at two time steps. The computation in each cell follows Eq. A.12: $\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t)$. The resulting probability expressed in each cell is Eq. A.11: $\alpha_t(j) = P(o_1, o_2 \ldots o_t, q_t = j|\lambda)$.

https://web.stanford.edu/~jurafsky/slp3/A.pdf

# Forward Algorithm Example



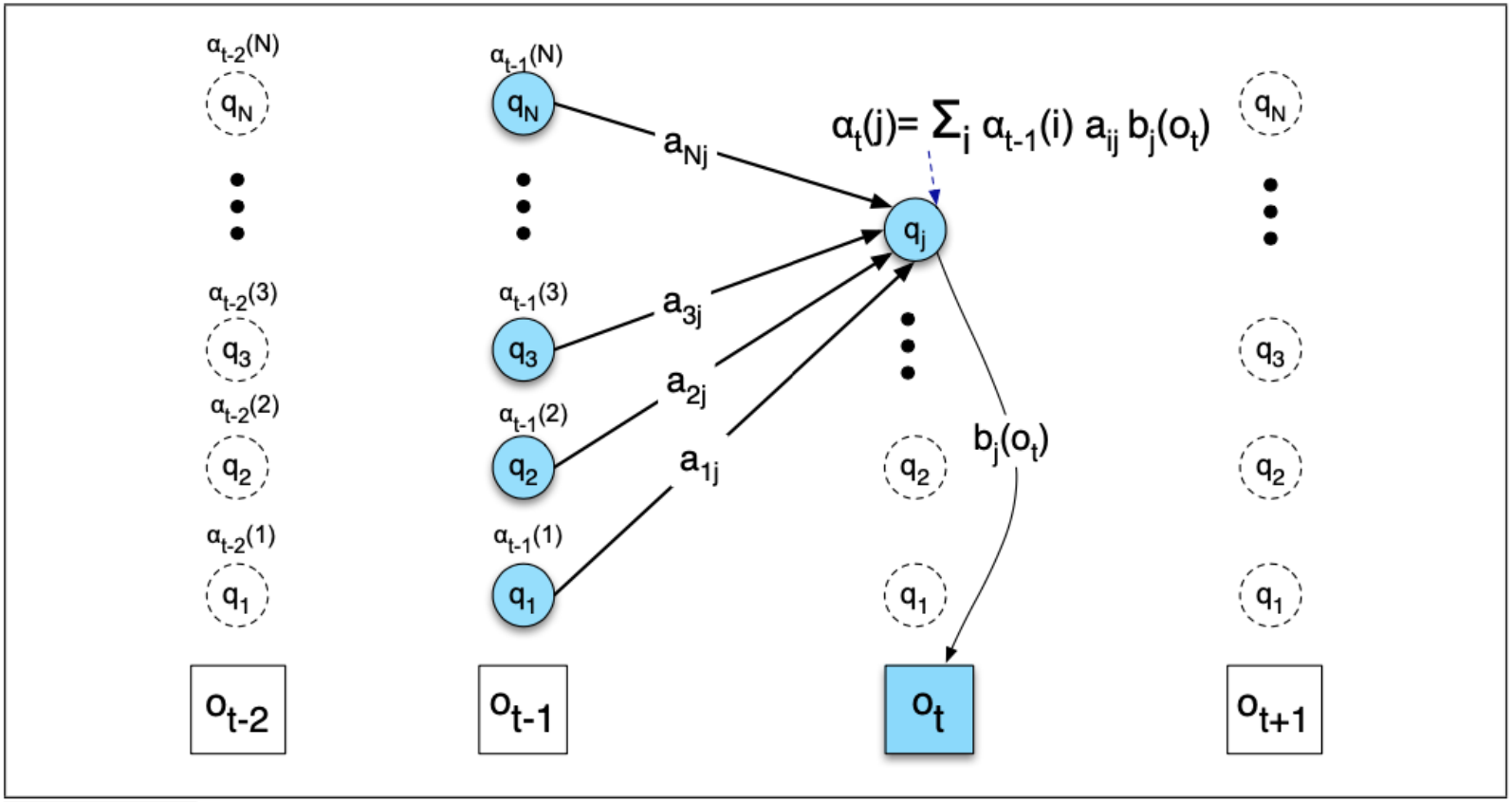
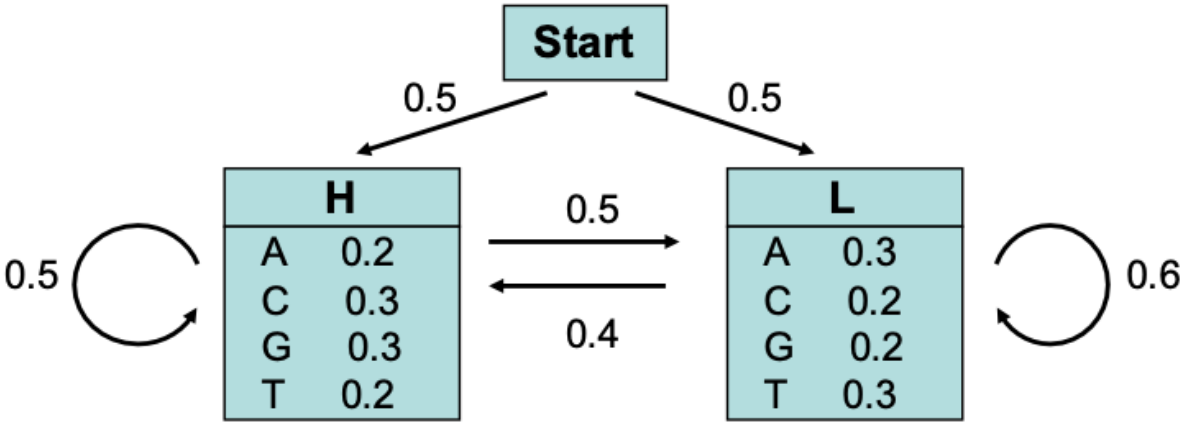



**Figure A.6** Visualizing the computation of a single element $\alpha_t(i)$ in the trellis by summing all the previous values $\alpha_{t-1}$, weighted by their transition probabilities $a$, and multiplying by the observation probability $b_i(o_t)$. For many applications of HMMs, many of the transition probabilities are 0, so not all previous states will contribute to the forward probability of the current state. Hidden states are in circles, observations in squares. Shaded nodes are included in the probability computation for $\alpha_t(i)$.

https://web.stanford.edu/~jurafsky/slp3/A.pdf

# Forward Algorithm Example





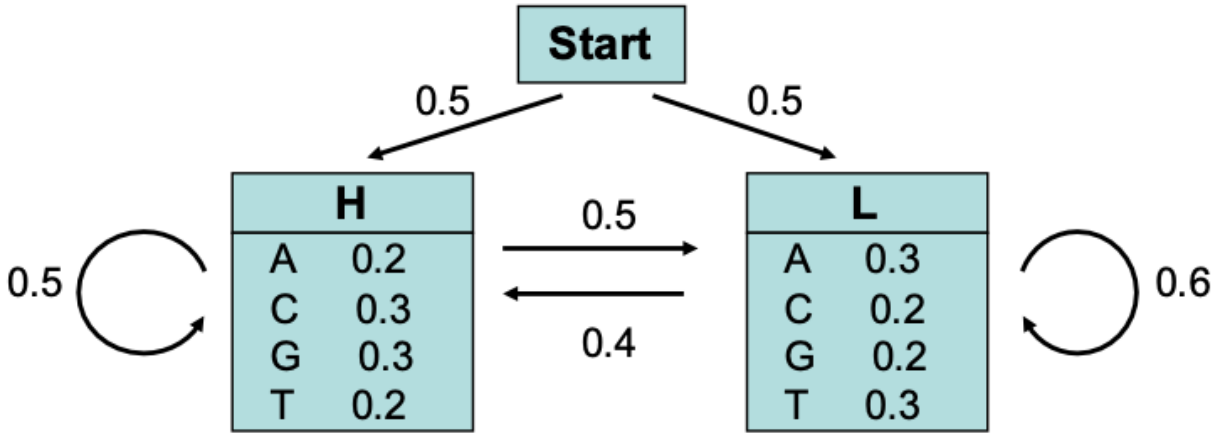

Let's consider the following simple HMM. This model is composed of 2 states, **H** (high GC content) and **L** (low GC content). We can for example consider that state H characterizes coding DNA while L characterizes non-coding DNA.

The model can then be used to predict the region of coding DNA from a given sequence.

Sources: For the theory, see Durbin *et al* (1998);
For the example, see Borodovsky & Ekisheva (2006), pp 80-81

# Forward Algorithm Example





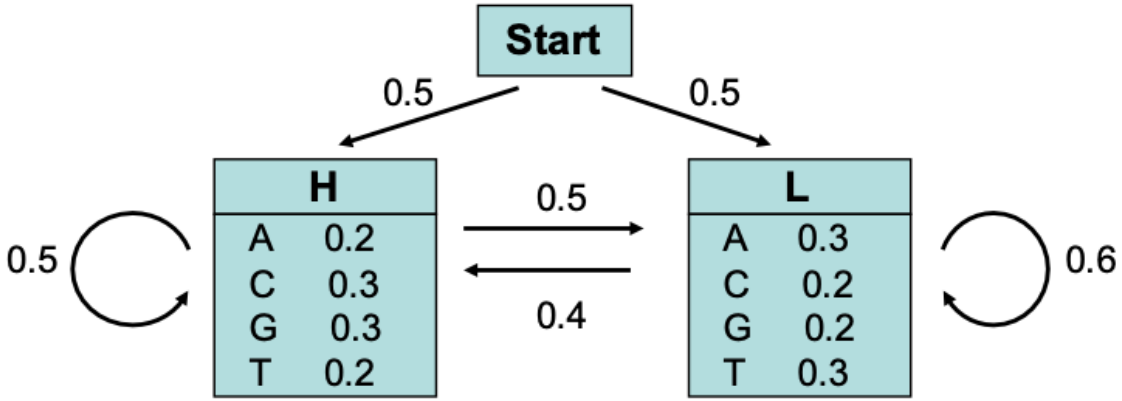

Consider now the sequence S= **GGCA**

What is the probability P(S) that this sequence S was generated by the HMM model?

This probability P(S) is given by the sum of the probabilities $p_i(S)$ of each possible path that produces this sequence.

# Forward Algorithm Example





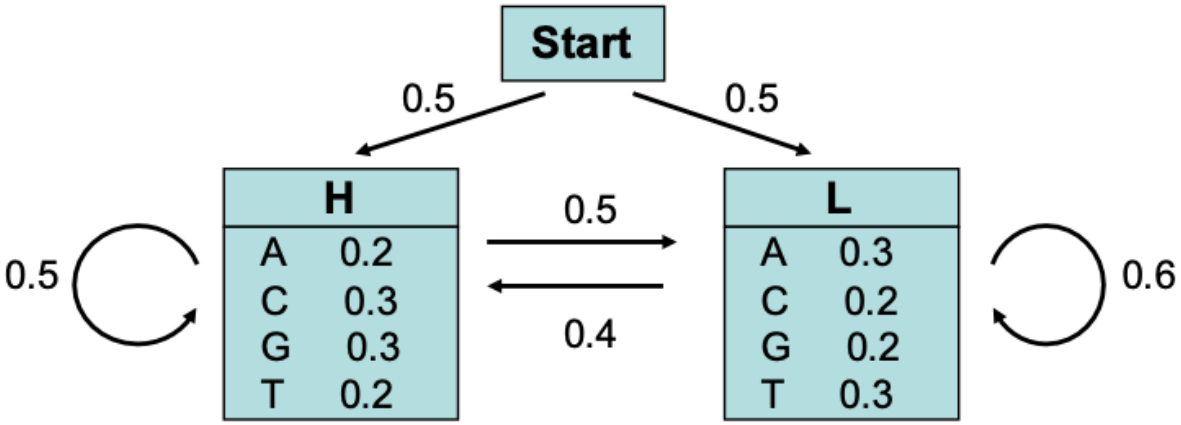

**Forward algorithm**

Consider now the sequence S= **GGCA**

| | Start | G | G | C | A |
|---|---|---|---|---|---|
| H | 0 | 0.5*0.3=0.15 | | | |
| L | 0 | 0.5*0.2=0.1 | | | |

# Forward Algorithm Example



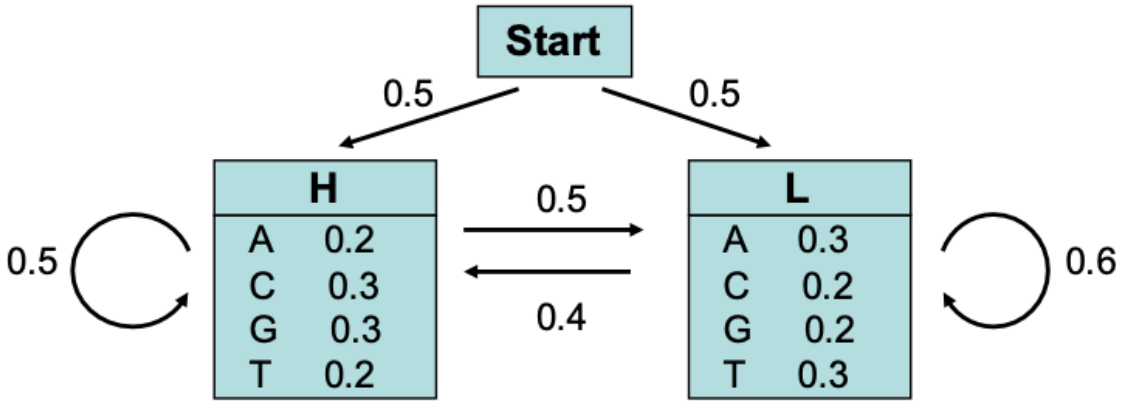



Consider now the sequence S= **GGCA**

| | Start | G | G | C | A |
|---|---|---|---|---|---|
| H | 0 | 0.5*0.3=0.15 → | 0.15*0.5*0.3 + 0.1*0.4*0.3=0.0345 | | |
| L | 0 | 0.5*0.2=0.1 → | | | |

# Forward Algorithm Example



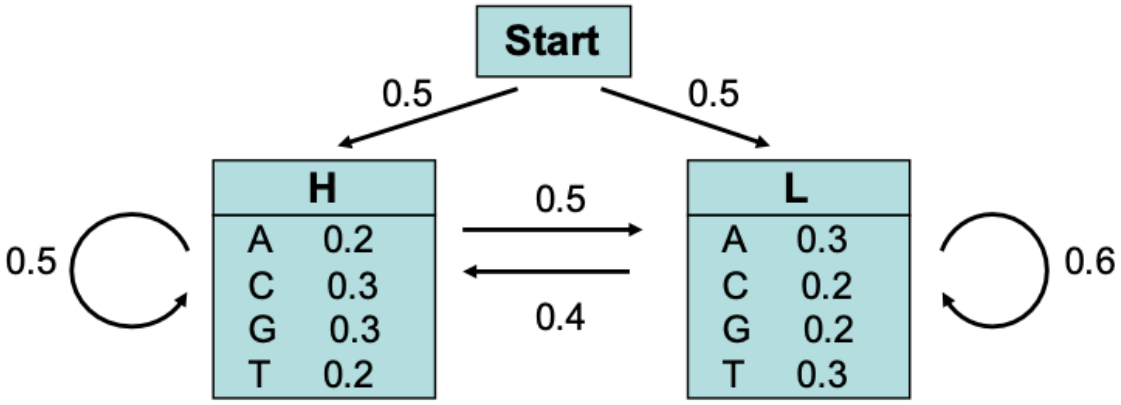



Consider now the sequence S= **GGCA**

| | Start | G | G | C | A |
|---|---|---|---|---|---|
| H | 0 | 0.5*0.3=0.15 | 0.15*0.5*0.3 + 0.1*0.4*0.3=0.0345 | | |
| L | 0 | 0.5*0.2=0.1 | 0.1*0.6*0.2 + 0.15*0.5*0.2=0.027 | | |

# Forward Algorithm Example



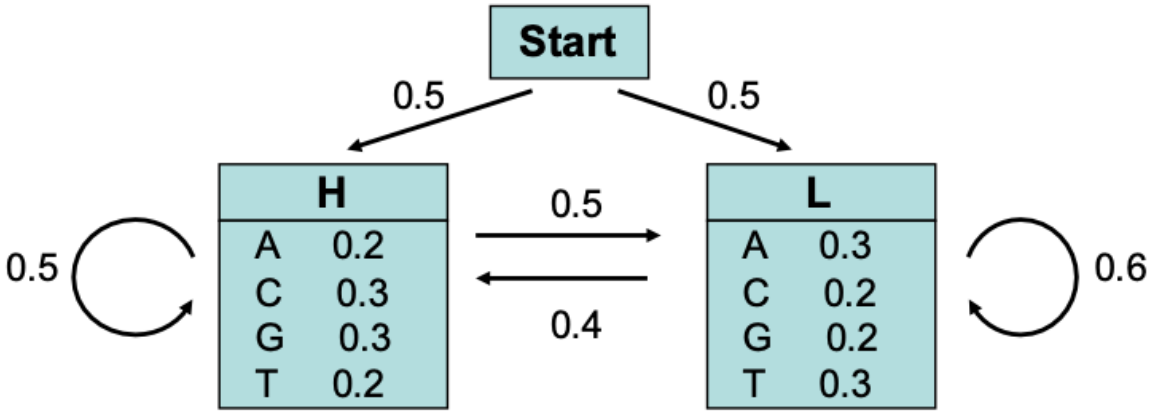



Consider now the sequence S= **GGCA**

| | Start | G | G | C | A |
|---|---|---|---|---|---|
| H | 0 | 0.5*0.3=0.15 | 0.15*0.5*0.3 + 0.1*0.4*0.3=0.0345 | ... + ... | |
| L | 0 | 0.5*0.2=0.1 | 0.1*0.6*0.2 + 0.15*0.5*0.2=0.027 | ... + ... | |

# Forward Algorithm Example






Consider now the sequence S= **GGCA**

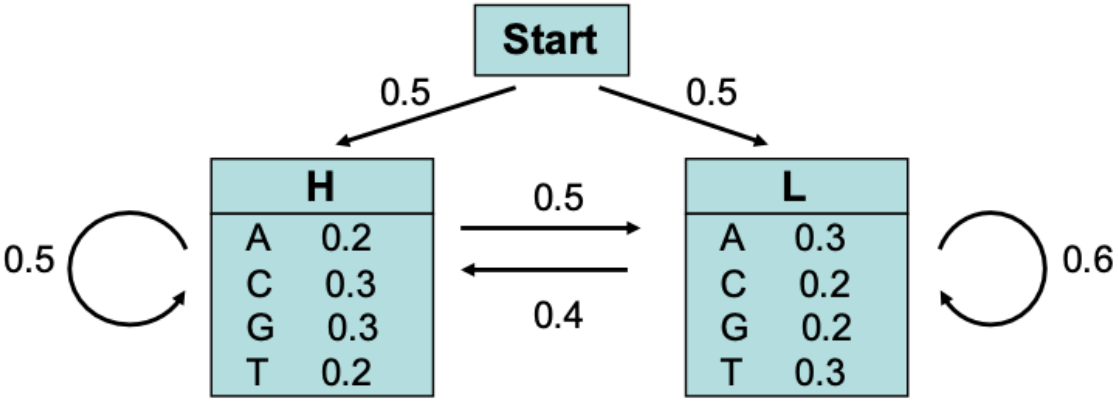
| | Start | G | G | C | A |
|---|---|---|---|---|---|
| H | 0 | 0.5*0.3=0.15 | 0.15*0.5*0.3 + 0.1*0.4*0.3=0.0345 | ... + ... | 0.0013767 |
| L | 0 | 0.5*0.2=0.1 | 0.1*0.6*0.2 + 0.15*0.5*0.2=0.027 | ... + ... | 0.0024665 |

=> The probability that the sequence S was generated by the HMM model is thus P(S)=0.0038432.

Σ = 0.0038432

# Forward Algorithm Example






The probability that sequence S="GGCA" was generated by the HMM model is $P_{HMM}(S) = 0.0038432$.

To assess the significance of this value, we have to compare it to the probability that sequence S was generated by the background model (i.e. by chance).

Ex: If all nucleotides have the same probability, $p_{bg}=0.25$; the probability to observe S by chance is: $P_{bg}(S) = p_{bg}^4 = 0.25^4 = 0.00396$.

Thus, for this particular example, it is likely that the sequence S does not match the HMM model ($P_{bg} > P_{HMM}$).

*NB: Note that this toy model is very simple and does not reflect any biological motif. If fact both states H and L are characterized by probabilities close to the background probabilities, which makes the model not realistic and not suitable to detect specific motifs.*

# Decoding

**Decoding over a single path:**

- **Input:**
  - A sequence of observations $x = [x_1, x_2, x_3 ...]$ generated by a HMM(Q, A, p, V, E)
- **Output:**
  - The most probably path of states $\pi^* = \pi^*_1, \pi^*_2, \pi^*_3 ....$

**Decoding over all paths:**

- **Input:**
  - A sequence of observations $x = [x_1, x_2, x_3 ...]$ generated by a HMM(Q, A, p, V, E)
- **Output:**
  - The path of states $\pi^* = \pi^*_1, \pi^*_2, \pi^*_3 ....$ that contains the most likely state at each time point.

# Single path decoding

**"Given some observed sequence, what path gives the maximum likelihood of observing this sequence?"**

- For the **single path decoding** problem, we could imagine a brute-force approach where we calculate the joint probabilities of a given emission sequence over all possible paths and then pick the path with the maximum joint probability.

- But in most systems there is an exponential number of possible paths and such a brute force approach is time consuming and impractical. **However, Dynamic Programming can be used to solve this problem.**

- We would like to find the most likely sequence of states based on the observation. The goal is to find π*, the sequence of hidden states that maximizes the joint probability with the given sequence of emissions:

$$\pi^* = \text{argmax}_\pi \; P(x, \pi)$$

# Viterbi Decoding



- Given the emitted sequence **x**, we can evaluate any path through hidden states. However, **we are looking for the best path.**

- **The best path through a given state must contain with it the following:**
  - The best path to previous state
  - The best transition from the previous state to this state
  - The best path to the end state

- Therefore, the nbest path can be obtained based on the best path of previous states, i.e., we can find a recurrence for the best path.

- **The Viterbi algorithm is a dynamic programming algorithm commonly used to obtain the best path.**

$$\pi^* = \text{argmax}_\pi \; P(x, \pi)$$

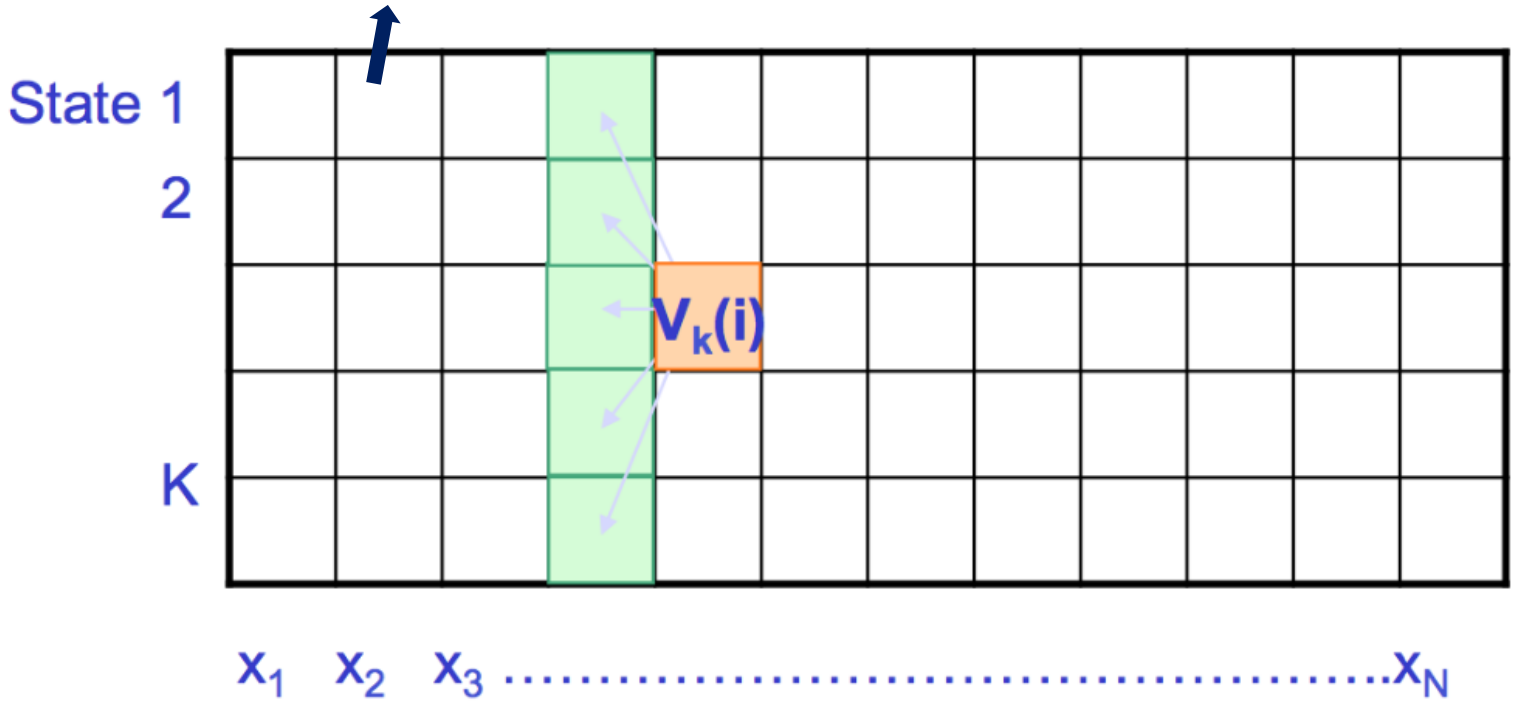
# Viterbi Decoding



The **Viterbi algorithm** is a dynamic programming approach to find the path with the maximum score over all paths. This algorithm defines $V_k(i)$ as the probability of the most likely path ending at position (or time instance) *i* in state *k*, for each *k* and recursively computes:

$$V_k(i) = e_k(x_i) * max_j(V_j(i-1)a_{jk})$$

*Each cell represents the probability that the HMM is in state k after seeing the first ith observations and passing through the most probable state sequence. The value of each cell is computed by recursively taking the most probable path that could lead us to this cell.*

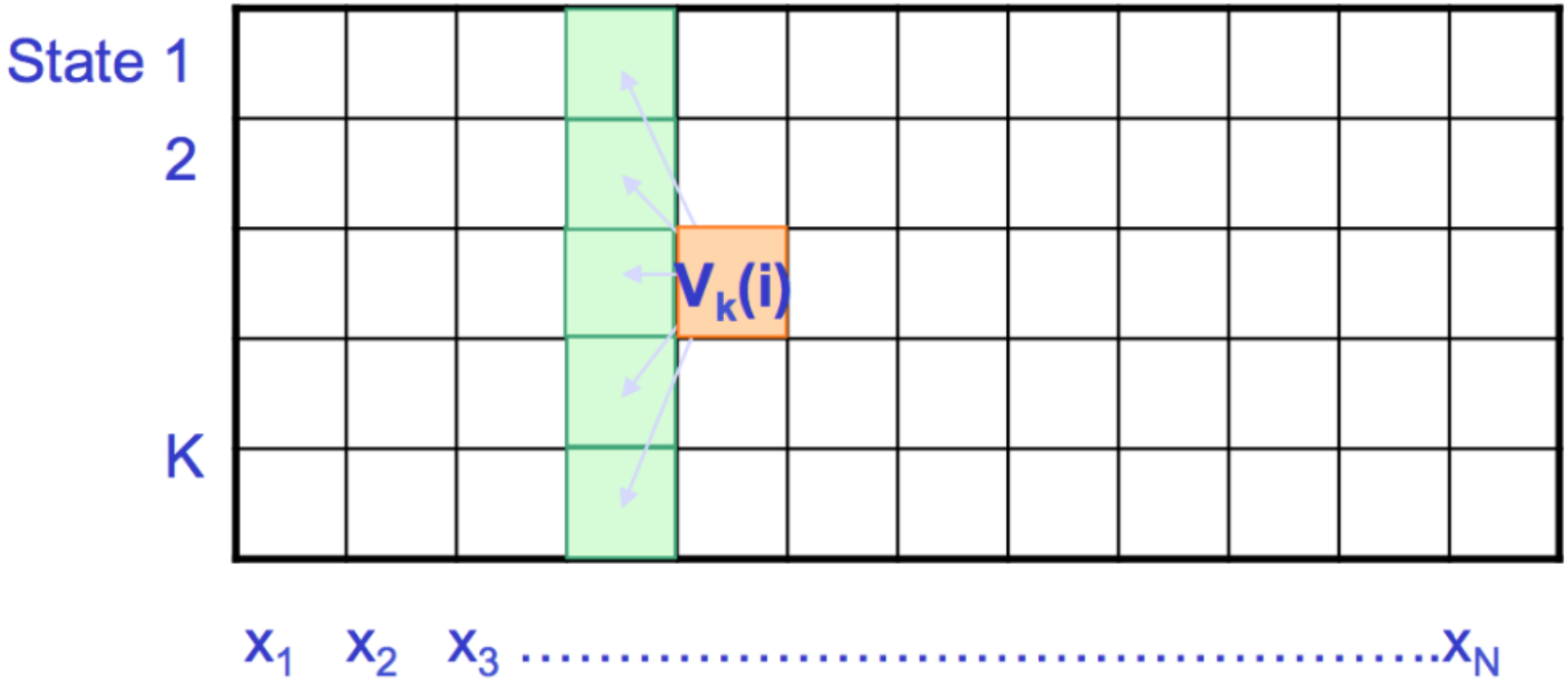



*$e_k$ is the emission probability for state k of emiting $x_i$ and $a_{jk}$ is the probability of transitioning from state j to state k*

# Viterbi Decoding



1. Initialization $(i = 0) : v_0(0) = 1, v_k(0) = 0 \text{ for } k > 0$
2. Recursion $(i = 1 \ldots N) : v_k(i) = e_k(x_i) \max_j (a_{jk} v_j(i-1)) ; ptr_i(l) = \arg\max_j (a_{jk} v_j(i-1))$
3. Termination: $P(x, \pi^*) = \max_k v_k(N); \pi_N^* = \arg\max_k v_k(N)$
4. Traceback $(i = N \ldots 1) : \pi_{i-1}^* = ptr_i(\pi_i^*)$




*$e_k$ is the emission probability for state k of emiting $x_i$ and $a_{jk}$ is the probability of transitioning from state j to state k*

# Viterbi Decoding Example

$$V_k(i) = e_k(x_i) * max_j(V_j(i-1)a_{jk})$$

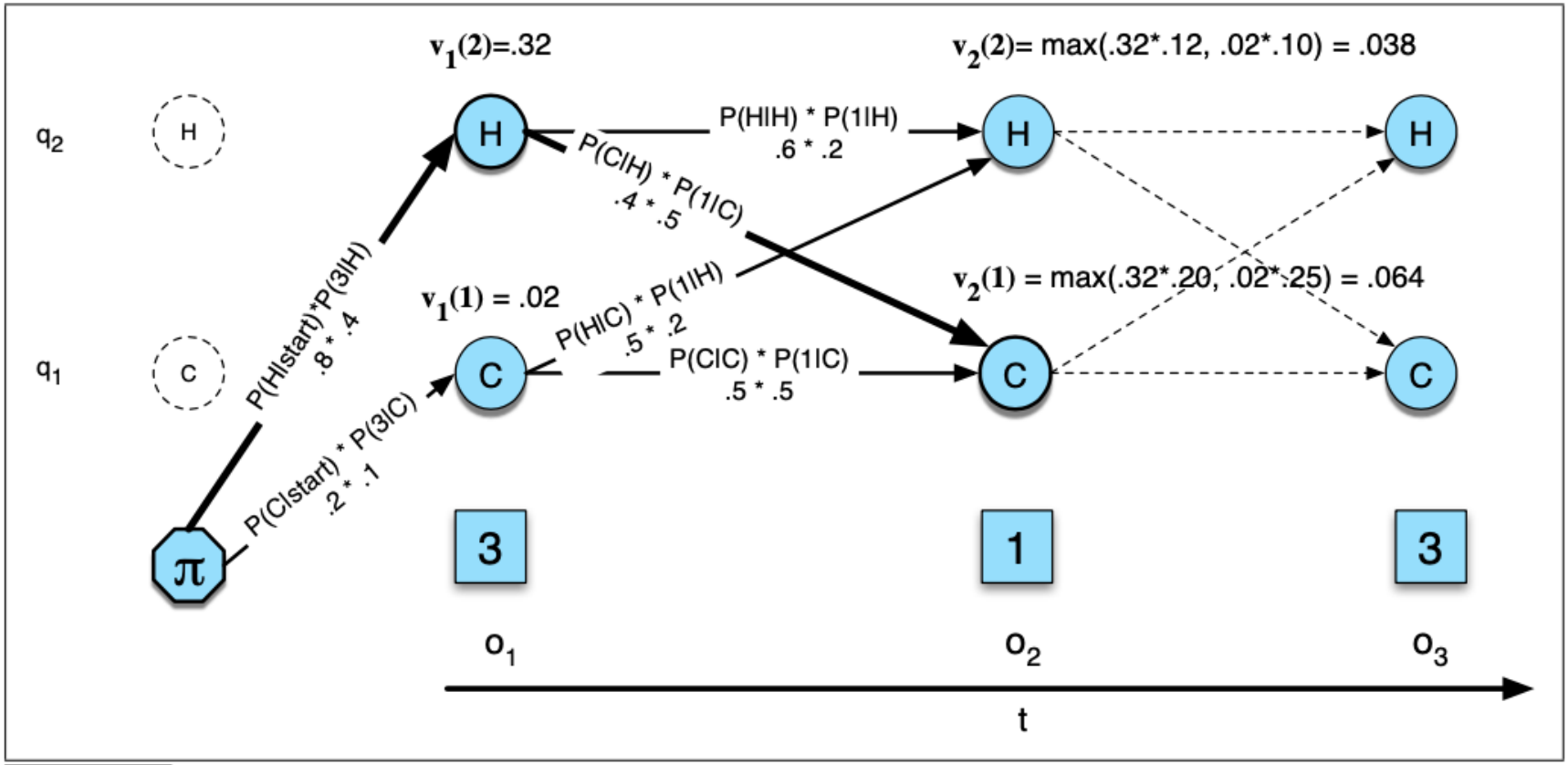



**Figure A.8** The Viterbi trellis for computing the best path through the hidden state space for the ice-cream eating events *3 1 3*. Hidden states are in circles, observations in squares. White (unfilled) circles indicate illegal transitions. The figure shows the computation of $v_t(j)$ for two states at two time steps. The computation in each cell follows Eq. A.14: $v_t(j) = \max_{1 \le i \le N-1} v_{t-1}(i)\, a_{ij}\, b_j(o_t)$. The resulting probability expressed in each cell is Eq. A.13: $v_t(j) = P(q_0, q_1, \ldots, q_{t-1}, o_1, o_2, \ldots, o_t, q_t = j|\lambda)$.

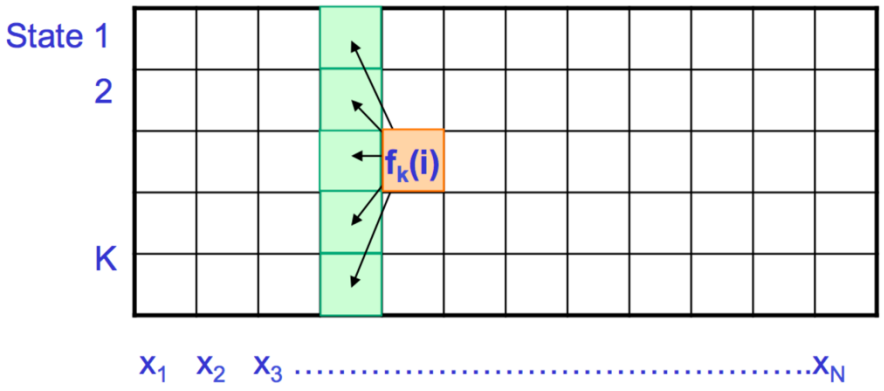
https://web.stanford.edu/~jurafsky/slp3/A.pdf

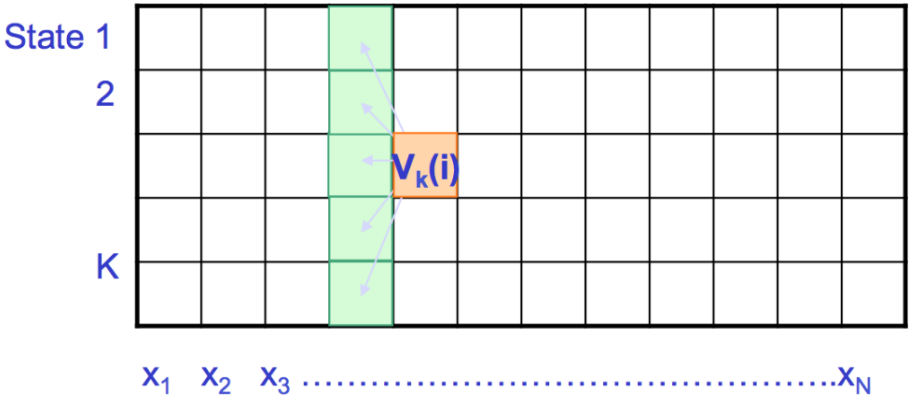
# Forward and Viterbi

- The Forward Algorithm and the Viterbi Algorith largely **share the same recursion**.
- The only difference lies in the fact that **the Viterbi algorithm**, seeking to find only the single most likely path, **uses a maximization function**, whereas the **Forward Algorithm**, seeking to find the total probability of the sequence over all paths, **uses a sum.**

$$f_k(i) = e_k(x_i) \sum_j f_j(i-1) a_{jk}$$

$$V_k(i) = e_k(x_i) * max_j(V_j(i-1) a_{jk})$$



# Forward and Viterbi

- The Viterbi algorithm also has one component that the foward algorithm does not: **backpointers.** The reason is that while the forward algorithm needs to produce an observation likelihood, **the Viterbi algorithm must produce a probability and also the most likely state sequence.** We compute this best sequence by **keeping track of the path of hidden states that led to each state** and then at the end **backtracking the best path to the beginning**.

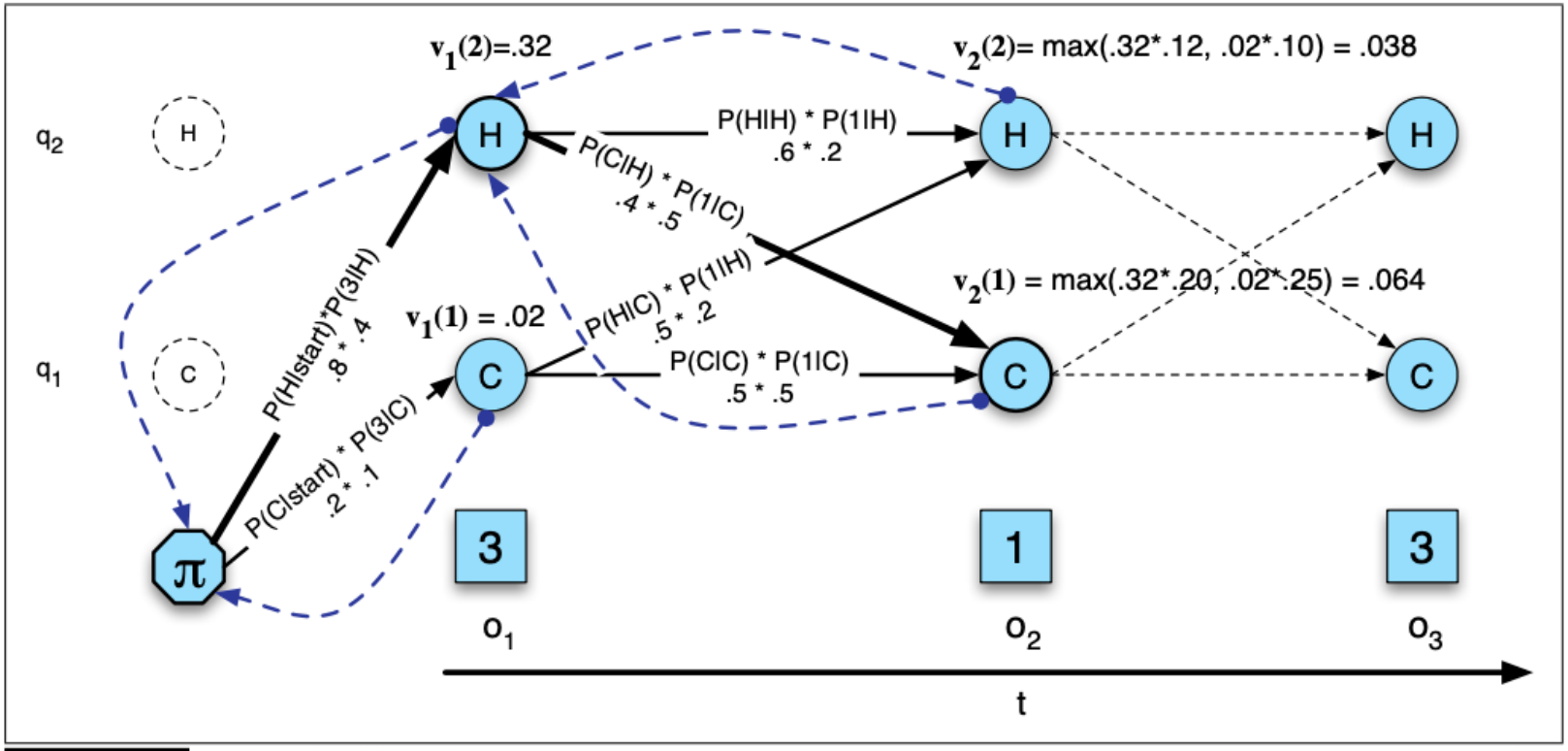



**Figure A.10** The Viterbi backtrace. As we extend each path to a new state account for the next observation, we keep a backpointer (shown with broken lines) to the best path that led us to this state.

# HMMER and HMMLEARN



- **HMMER** is a software suite that uses Hidden Markov Models to search for sequence homology, and making sequence alignments. It's maintained by Sean Eddy's lab and widely used together with a profile database like Pfam (*http://eddylab.org/software/hmmer/Userguide.pdf*)

- **hmmlearn** is a Python package for unsupervised learning and inference of Hidden Markov Models (*for general applications*). Built on top of scikit-learn, numpy, scipy, and matplotlib (*https://hmmlearn.readthedocs.io/en/latest/*). hmmlearn alows us to simulate sequences from an HMM, train an HMM from data, decode hidden state paths, and compute log-likelihoods.